

Bayesian Analysis Project

T Kiran Kumar

2024-11-25

R Markdown

This is an R Markdown document.

Aim:

1. To find out what attributes make a movie popular
2. Learning something new about movies For this, develop a Bayesian regression model to predict audience_score from the explanatory variables.

DATA

Data Collection: Acquired data about how much audiences and critics like movies as well as numerous other variables about the movies. It includes information from Rotten Tomatoes and IMDB for a random sample of movies.

Audience score needs to be predicted from sixteen explanatory variables, of which some of them are given in the original data set and the rest needs to be constructed.

Part 1: Data

Reasoning for generalizability: The present data is collected using simple random sampling (or stratified sampling). It is difficult to say we can generalize this, because collected data is minimal and lacks information in many ways, for example represent only two movie review sites.

Causality: No Random assignment was used. Hence causality can not be inferred. These are only observational data and no experiment is done, causality might not be attributed. (Ref. open-intro statistics, chapter - 1)

Load the necessary Libraries

```
library(ggplot2)
library(dplyr)
library(statsr)
library(BAS)
```

Part 2: Data manipulation

Load the given movies Data and have a look at various parameters available...

```
load("movies.Rdata")
str(movies)
```

```
## tibble [651 × 32] (S3: tbl_df/tbl/data.frame)
## $ title           : chr [1:651] "Filly Brown" "The Dish" "Waiting for Guffman" "The Age of Innocence" ...
## $ title_type      : Factor w/ 3 levels "Documentary",...: 2 2 2 2 2 1 2 2 1 2 ...
## $ genre           : Factor w/ 11 levels "Action & Adventure",...: 6 6 4 6 7 5 6 6 5 6 ...
## $ runtime         : num [1:651] 80 101 84 139 90 78 142 93 88 119 ...
## $ mpaa_rating     : Factor w/ 6 levels "G","NC-17","PG",...: 5 4 5 3 5 6 4 5 6 6 ...
## $ studio          : Factor w/ 211 levels "20th Century Fox",...: 91 202 167 34 13 163 147 118 88 84 ...
## $ thtr_rel_year   : num [1:651] 2013 2001 1996 1993 2004 ...
## $ thtr_rel_month   : num [1:651] 4 3 8 10 9 1 1 11 9 3 ...
## $ thtr_rel_day     : num [1:651] 19 14 21 1 10 15 1 8 7 2 ...
## $ dvd_rel_year     : num [1:651] 2013 2001 2001 2001 2005 ...
## $ dvd_rel_month    : num [1:651] 7 8 8 11 4 4 2 3 1 8 ...
## $ dvd_rel_day      : num [1:651] 30 28 21 6 19 20 18 2 21 14 ...
## $ imdb_rating      : num [1:651] 5.5 7.3 7.6 7.2 5.1 7.8 7.2 5.5 7.5 6.6 ...
## $ imdb_num_votes   : int [1:651] 899 12285 22381 35096 2386 333 5016 2272 880 1249 6 ...
## $ critics_rating   : Factor w/ 3 levels "Certified Fresh",...: 3 1 1 1 3 2 3 3 2 1 ...
## $ critics_score    : num [1:651] 45 96 91 80 33 91 57 17 90 83 ...
## $ audience_rating  : Factor w/ 2 levels "Spilled","Upright": 2 2 2 2 1 2 2 1 2 2 ...
## $ audience_score   : num [1:651] 73 81 91 76 27 86 76 47 89 66 ...
## $ best_pic_nom     : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...
## $ best_pic_win      : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...
## $ best_actor_win    : Factor w/ 2 levels "no","yes": 1 1 1 2 1 1 1 2 1 1 ...
## $ best_actress_win  : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...
## $ best_dir_win      : Factor w/ 2 levels "no","yes": 1 1 1 2 1 1 1 1 1 1 ...
## $ top200_box        : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...
## $ director         : chr [1:651] "Michael D. Olmos" "Rob Sitch" "Christopher Guest" "Martin Scorsese" ...
## $ actor1           : chr [1:651] "Gina Rodriguez" "Sam Neill" "Christopher Guest" "Daniel Day-Lewis" ...
## $ actor2           : chr [1:651] "Jenni Rivera" "Kevin Harrington" "Catherine O'Hara" "Michelle Pfeiffer" ...
## $ actor3           : chr [1:651] "Lou Diamond Phillips" "Patrick Warburton" "Parker Posey" "Winona Ryder" ...
## $ actor4           : chr [1:651] "Emilio Rivera" "Tom Long" "Eugene Levy" "Richard E. Grant" ...
## $ actor5           : chr [1:651] "Joseph Julian Soria" "Genevieve Mooy" "Bob Balaban" "Alec McCowen" ...
## $ imdb_url         : chr [1:651] "http://www.imdb.com/title/tt1869425/" "http://www.imdb.com/title/tt0205873/" "http://www.imdb.com/title/tt0118111/" "http://www.imdb.com/title/tt0106226/" ...
## $ rt_url           : chr [1:651] "http://www.rottentomatoes.com/m/filly_brown_2012/" "http://www.rottentomatoes.com/m/dish/" "http://www.rottentomatoes.com/m/waiting_for_guffman/" "http://www.rottentomatoes.com/m/age_of_innocence/" ...
```

Create new variable based on title_type: New variable should be called feature_film with levels yes (movies that are feature films) and no

```
movies<-mutate(movies, feature_film=ifelse(title_type=='Feature Film', 'yes', 'no'))
movies$feature_film <-as.factor(movies$feature_film)
```

Create new variable based on genre: New variable should be called drama with levels yes (movies that are dramas) and no

```
movies <- mutate(movies,drama=ifelse(genre=='Drama', 'yes', 'no'))
movies$drama <- as.factor(movies$drama)
```

Create new variable based on mpaa_rating: New variable should be called mpaa_rating_R with levels yes (movies that are R rated) and no

```
movies <- mutate(movies, mpaa_rating_R = ifelse(mpaa_rating == 'R', 'yes', 'no'))
movies$mpaa_rating_R <- as.factor(movies$mpaa_rating_R)
```

Create two new variables based on thtr_rel_month:

1. New variable called oscar_season with levels yes (if movie is released in October, November, or December) and no

```
movies <- mutate(movies, oscar_season = ifelse(thtr_rel_month >= 10, 'yes', 'no'))
movies$oscar_season <- as.factor(movies$oscar_season)
```

2. New variable called summer_season with levels yes (if movie is released in May, June, July, or August) and no

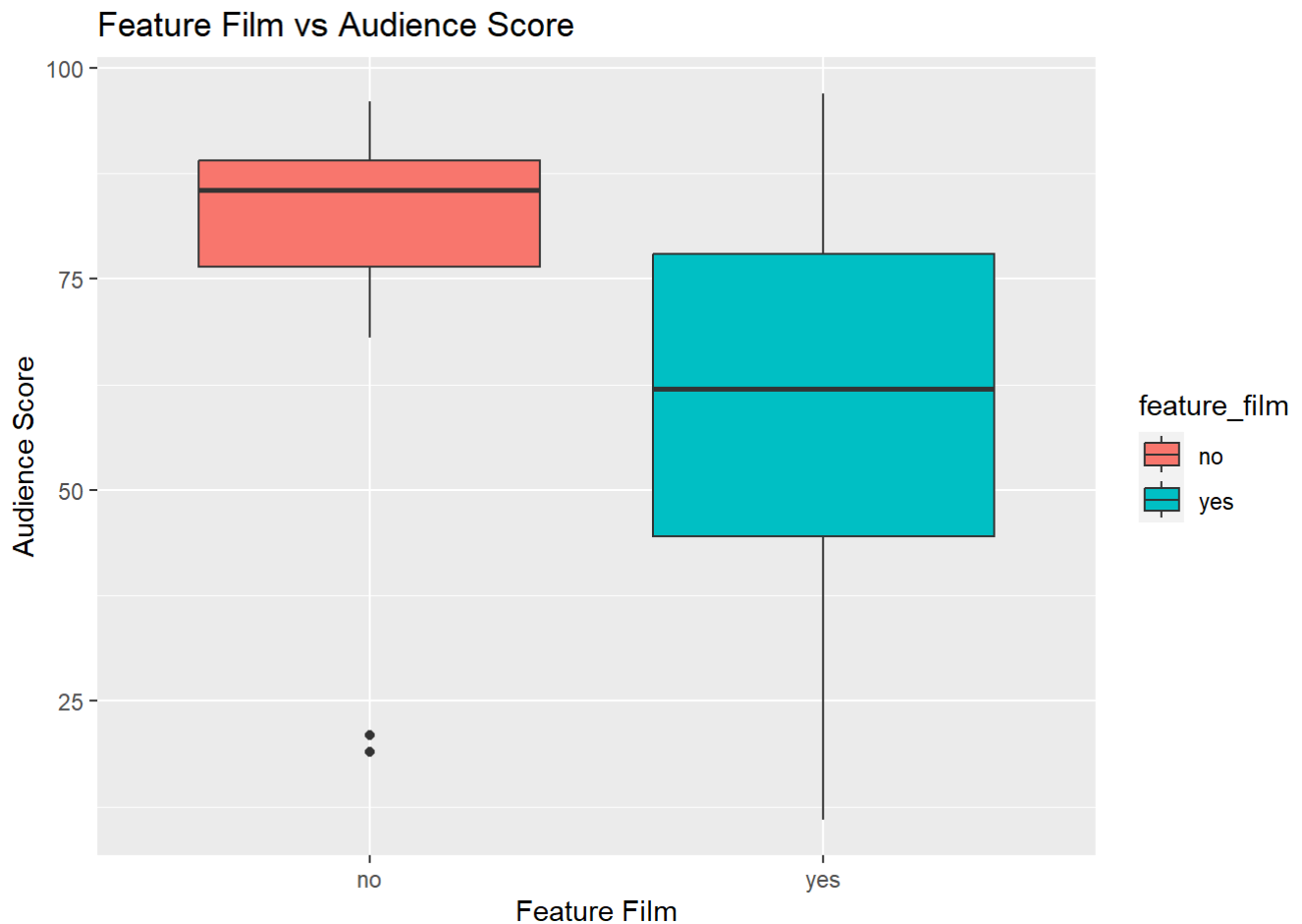
```
movies <- mutate(movies, summer_season = ifelse(thtr_rel_month %in% 5:8, 'yes', 'no'))
movies$summer_season <- as.factor(movies$summer_season)
```

Part 3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is a key part of analysis of given data.

We start out every analysis with EDA to familiarize ourselves with the data. First, we want to check to see if some of the samples or experiments produce unusable data and take them out of the analysis. We also perform EDA at the end to check for nonsensical results. In fact, one should perform EDA all throughout the analysis. For example, before applying any of the techniques we have learned, we want to make sure the data are in agreement with the necessary assumptions. (From edX: Statistics and R)

```
# Plot of Audience score Vs Feature Film
ggplot(aes(y = audience_score, x = feature_film, fill = feature_film), data = movies)
+ geom_boxplot() +
  labs(title = "Feature Film vs Audience Score", x = "Feature Film", y = "Audience Score")
```



The audience score is high for non-feature films relative to feature films. Couple of outliers are present in non-feature films

```
sum(movies$feature_film == "yes")
```

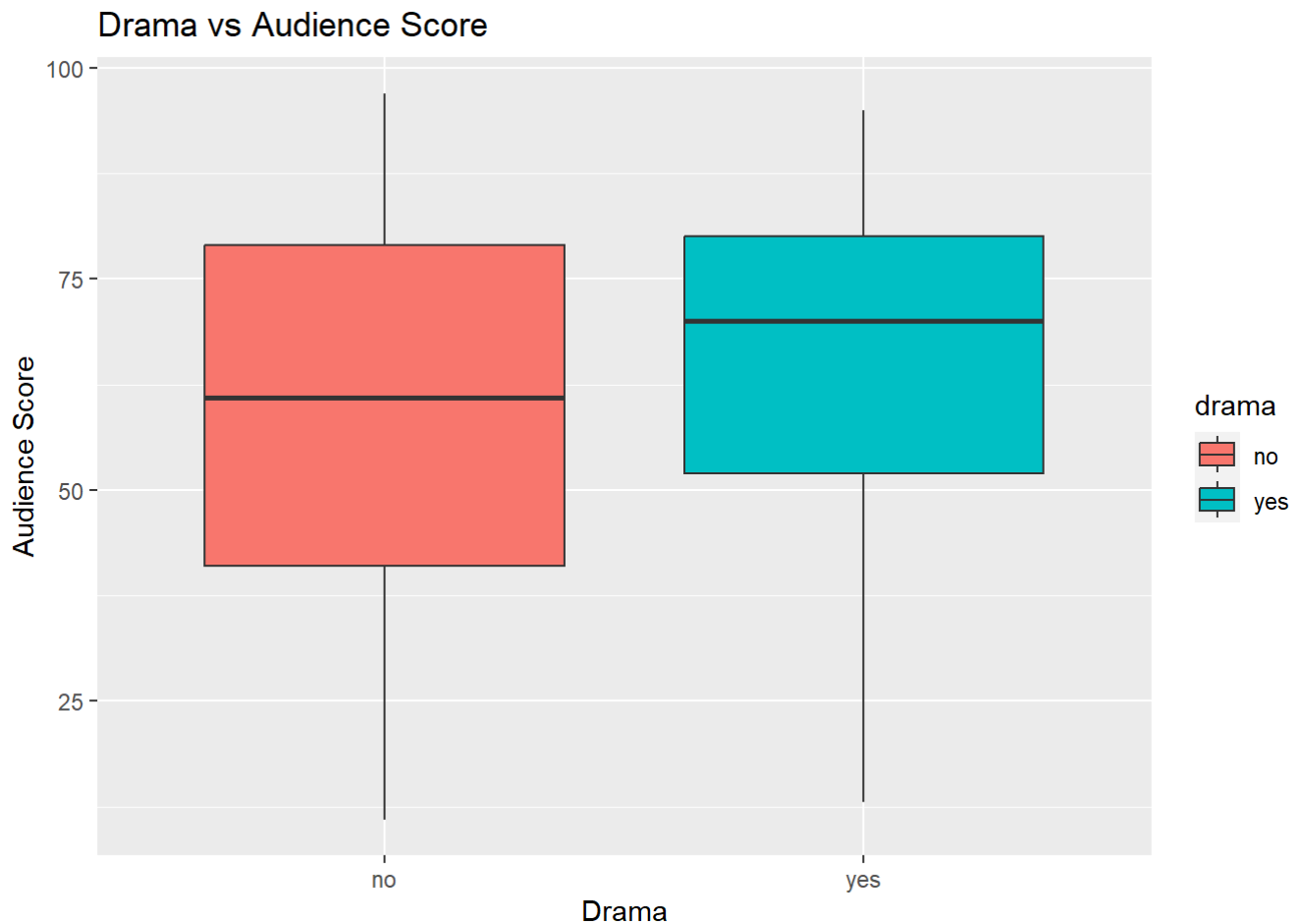
```
## [1] 591
```

```
sum(movies$feature_film == "no")
```

```
## [1] 60
```

But, we can see that non-feature film data is only one tenth of feature film (591 Vs. 60)

```
# Plot of Audience score Vs Drama
ggplot(aes(y = audience_score, x = drama, fill = drama), data = movies) + geom_boxplot() +
  labs(title = "Drama vs Audience Score", x = "Drama", y = "Audience Score")
```



The audience score is similar for both drama and non-drama movies, though drama movies audience_score is bit high.

```
sum(movies$drama == "yes" )
```

```
## [1] 305
```

```
sum(movies$drama == "no" )
```

```
## [1] 346
```

The number of drama movies in the data are comparable to number of non-drama movies (305 Vs. 346)

```
# Plot of Audience score Vs mpaa_rating_R
ggplot(aes(y = audience_score, x = mpaa_rating_R, fill = mpaa_rating_R), data = movies) +
  geom_boxplot() +
  labs(title = "MPAA Rating vs Audience Score", x = "Mpaa Rating R", y = "Audience Score")
```



The audience score is almost similar for both R-Rated and non-R-Rated movies,

```
sum(movies$mpaa_rating_R == "yes")
```

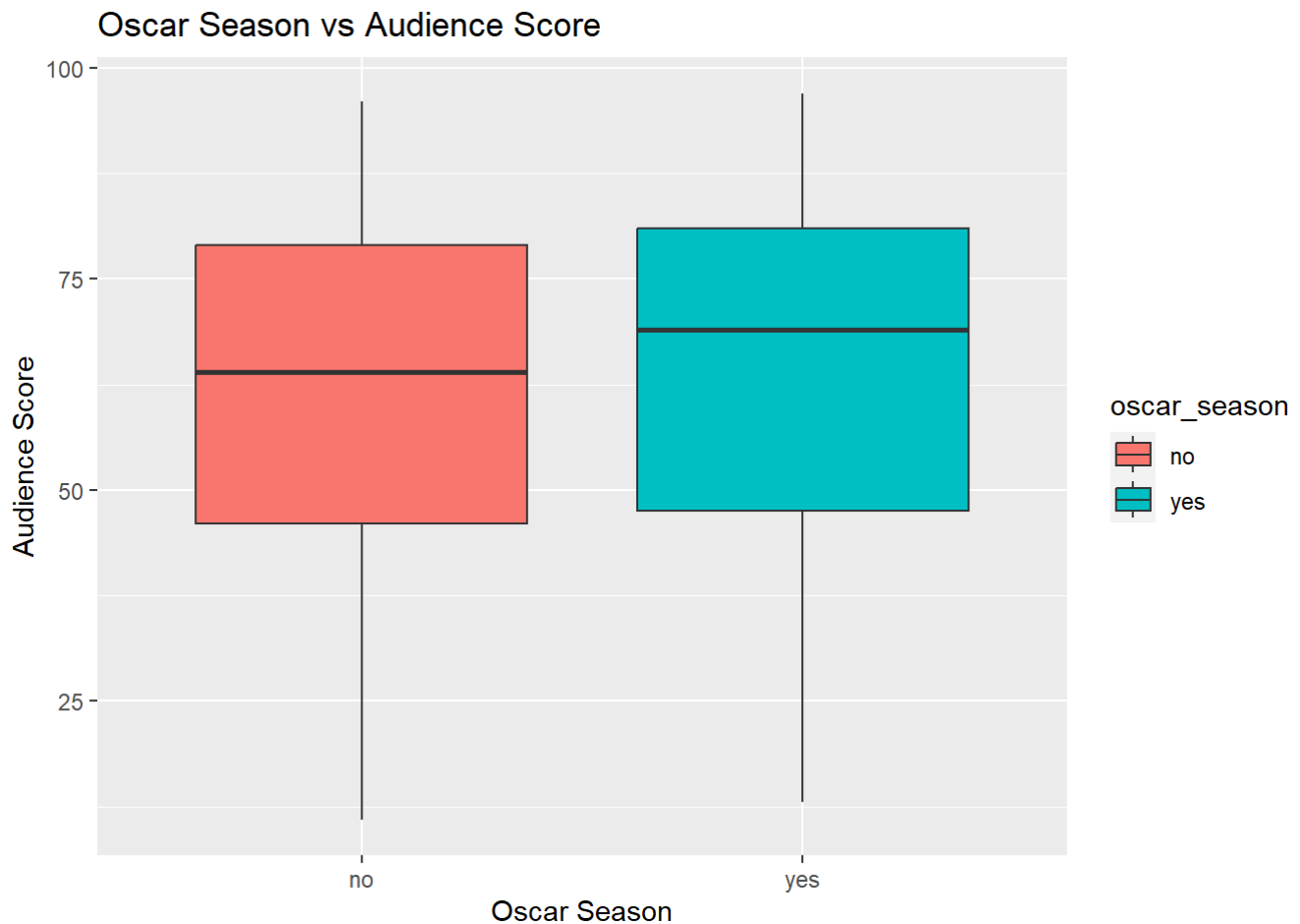
```
## [1] 329
```

```
sum(movies$mpaa_rating_R == "no")
```

```
## [1] 322
```

The number of R-Rated movies in the data are almost the same as number of non-R-Rated movies (329 Vs. 322)

```
# Plot of Audience score Vs oscar_season
ggplot(aes(y = audience_score, x = oscar_season, fill = oscar_season), data = movies)
+ geom_boxplot() +
  labs(title = "Oscar Season vs Audience Score", x = "Oscar Season", y = "Audience Score")
```



The audience score is slightly higher for the movies released in oscar season than the ones released in non-oscar season.

```
sum(movies$oscar_season == "yes")
```

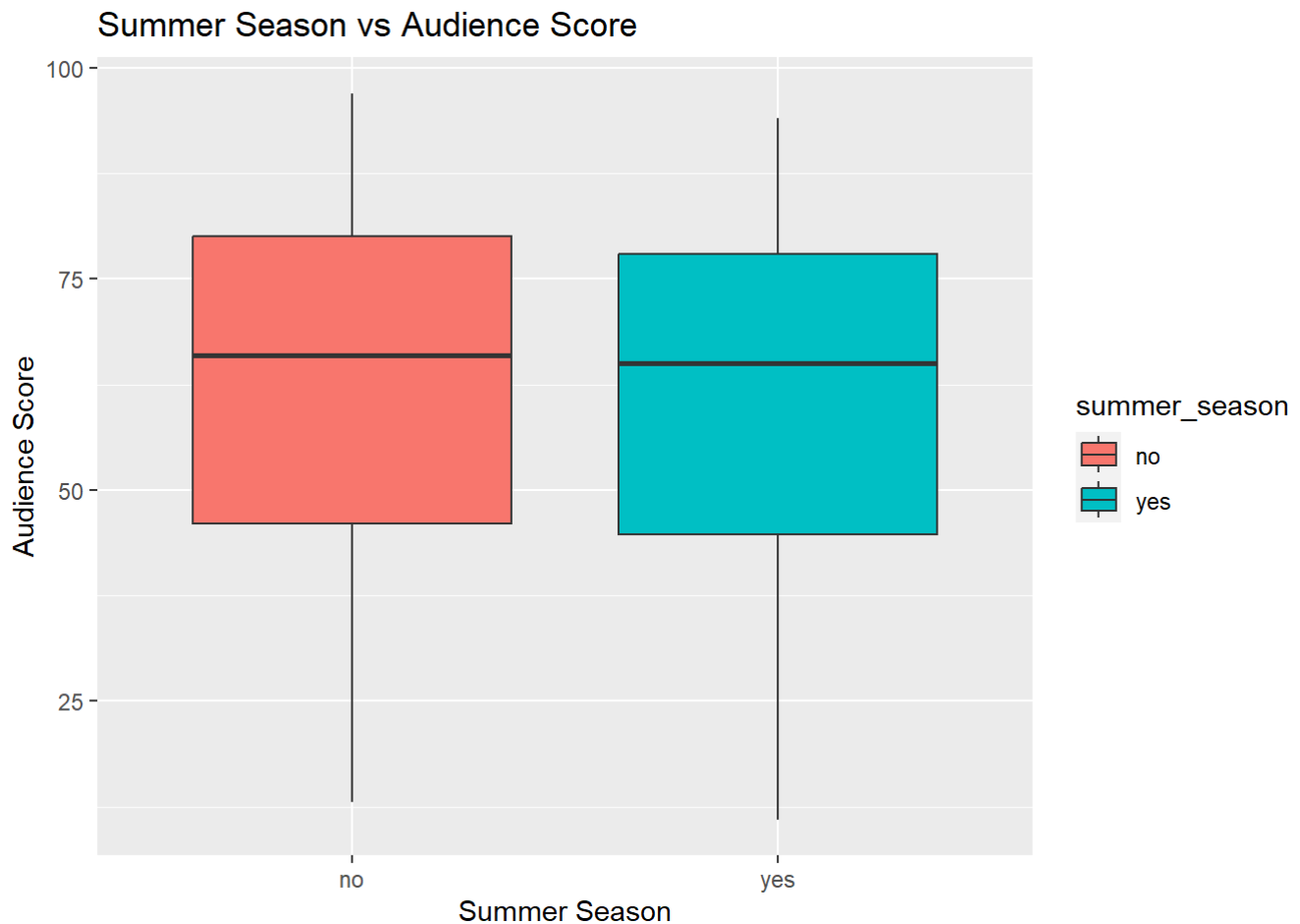
```
## [1] 191
```

```
sum(movies$oscar_season == "no")
```

```
## [1] 460
```

In the data, the number of oscar_season movies is 191, but the number of non-Oscar_season movies is 460, more than double of oscar_season movies

```
# Plot of Audience score Vs summer_season
ggplot(aes(y = audience_score, x = summer_season, fill = summer_season), data = movies) +
  geom_boxplot() +
  labs(title = "Summer Season vs Audience Score", x = "Summer Season", y = "Audience Score")
```



The audience score is almost the same for the movies released in summer and non-summer seasons

```
sum(movies$summer_season == "yes")
```

```
## [1] 208
```

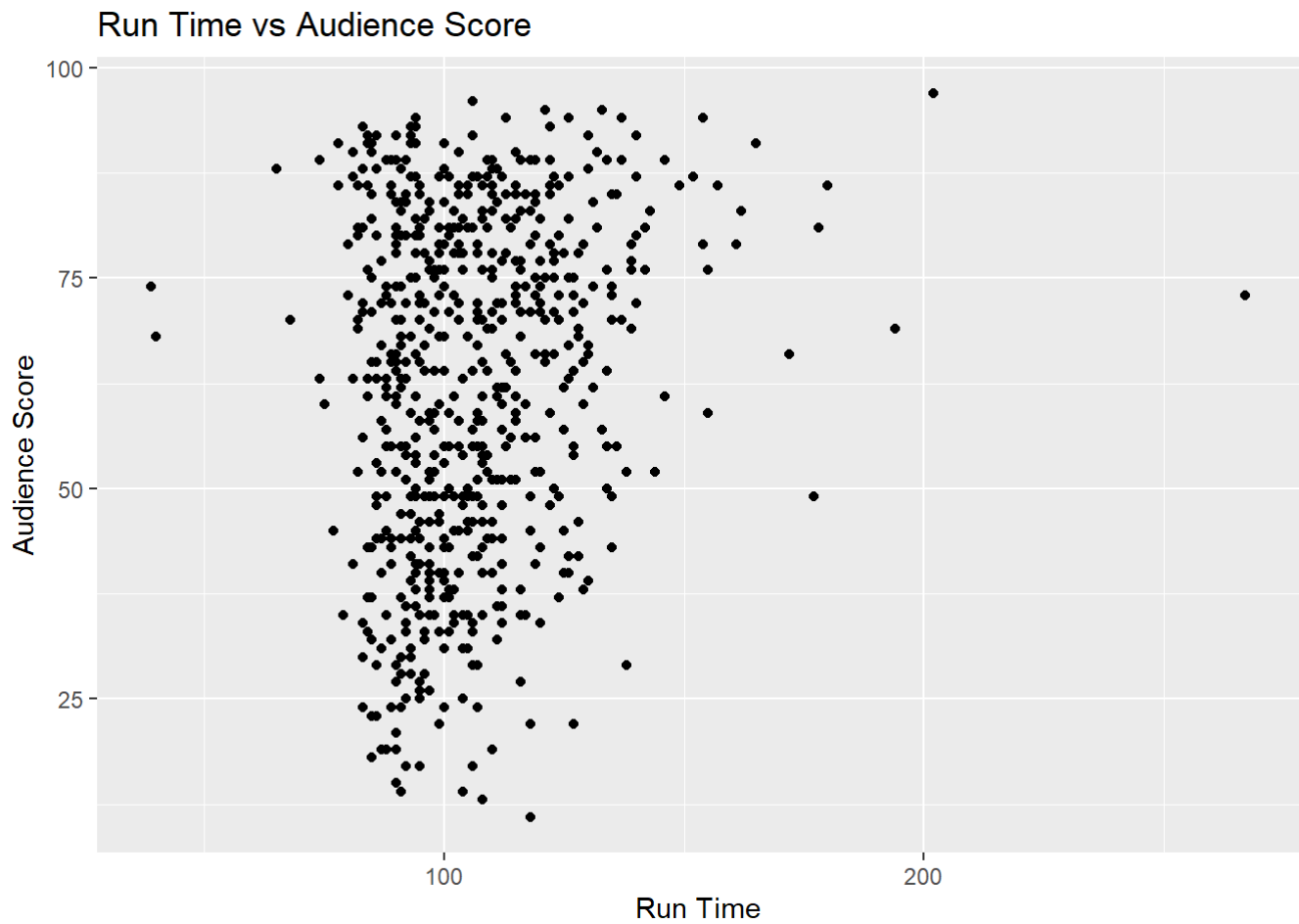
```
sum(movies$summer_season == "no")
```

```
## [1] 443
```

In the data, the number of summer_season movies is 208, but the number of non-summer_season movies is 443, more than double of summer_season movies

```
# Plot runtime Vs audience_score
ggplot(movies, aes(x = runtime , y = audience_score)) +
  geom_point() +
  labs(title = "Run Time vs Audience Score", x = "Run Time", y = "Audience Score")
```

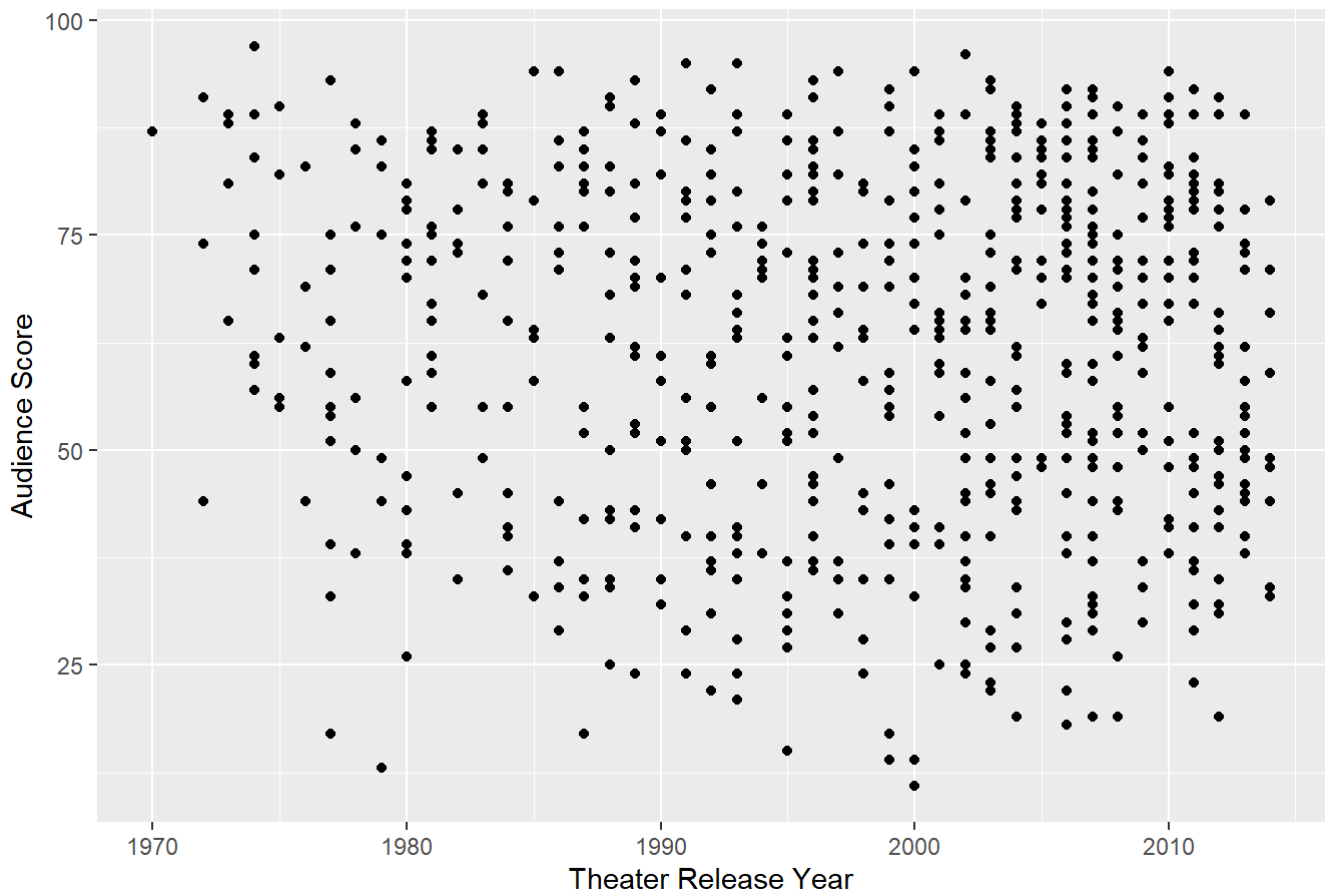
```
## Warning: Removed 1 rows containing missing values (`geom_point()`).
```



It appears from the graph that no specific relation exists between runtime and audience_score. However, for the runtime between 150 and 200 minutes, there appears to be high audience score for majority of the movies.

```
# Plot of Audience score Vs thtr_rel_year
ggplot(movies, aes(x = thtr_rel_year, y = audience_score)) +
  geom_point() +
  labs(title = "Theater Release Year vs Audience Score", x = "Theater Release Year", y
    = "Audience Score")
```

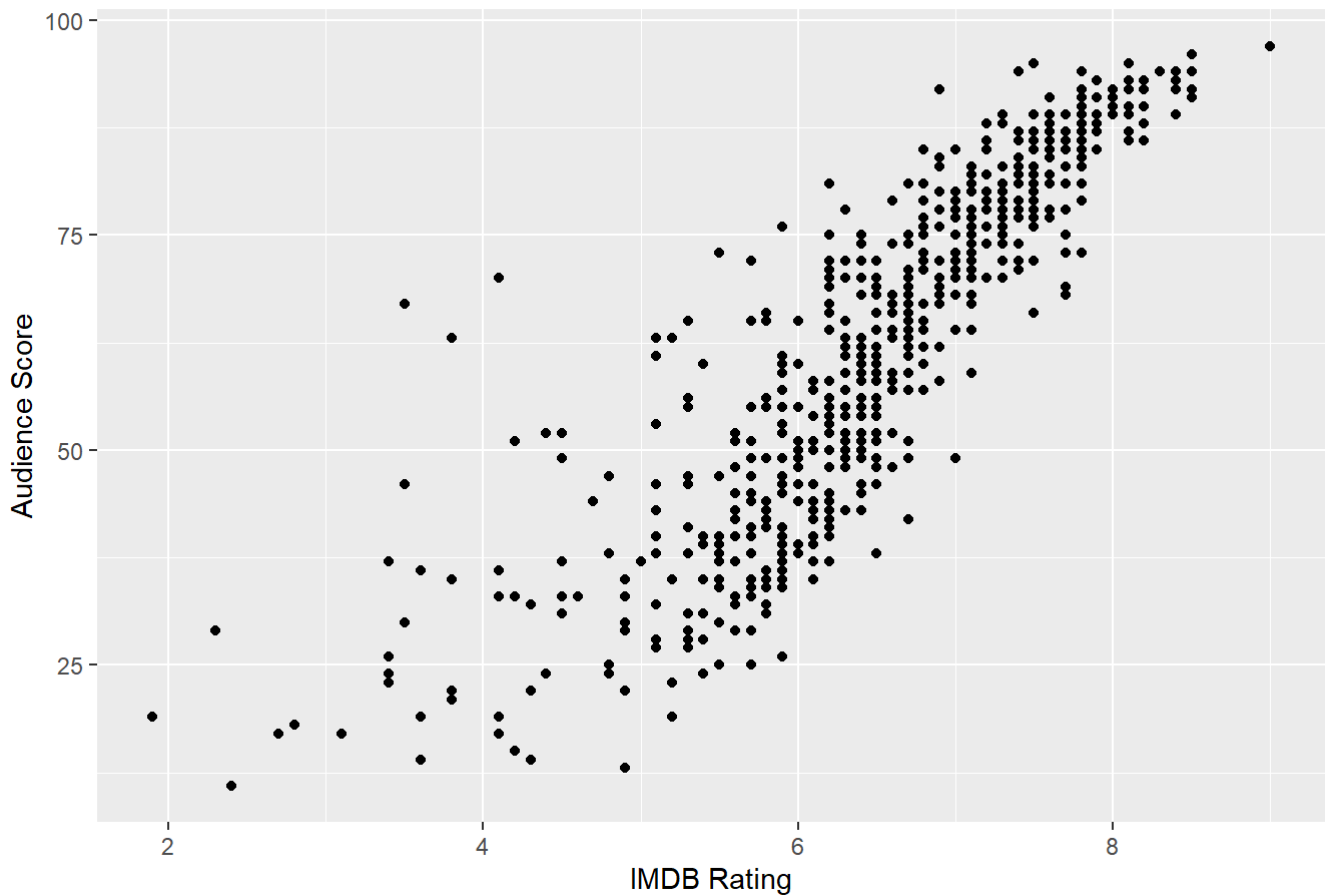
Theater Release Year vs Audience Score



Looks like that there is no particular association of the audience score with thtr_rel_year

```
# Plot imdb_rating Vs audience_score
ggplot(movies, aes(x = imdb_rating, y = audience_score)) +
  geom_point() +
  labs(title = "IMDB Rating vs Audience Score", x = "IMDB Rating", y = "Audience Score"
)
```

IMDB Rating vs Audience Score



Appears to have association between `imdb_rating` and `audience_score`. The correlation coefficient is given by:

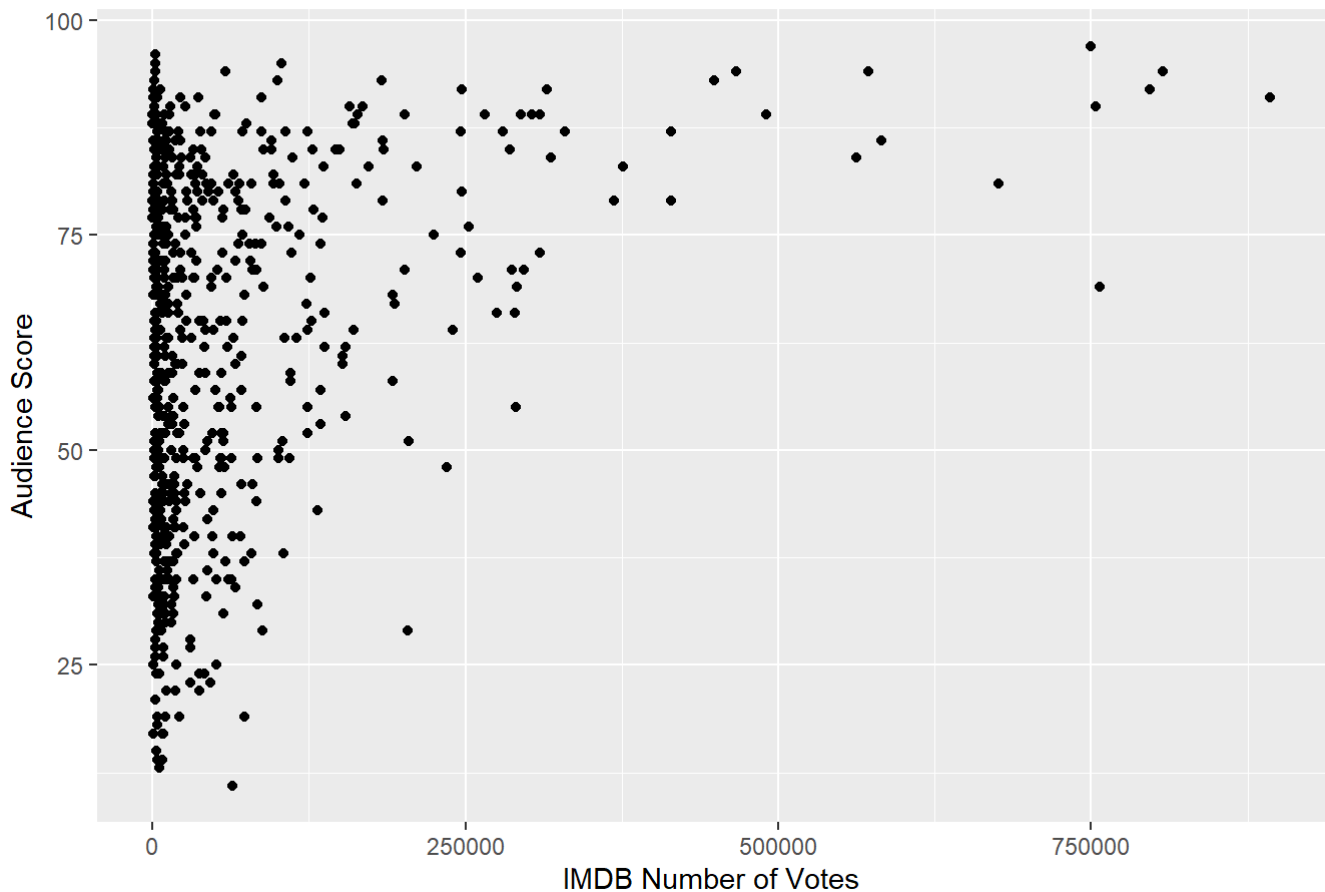
```
cor(movies$imdb_rating, movies$audience_score)
```

```
## [1] 0.8648652
```

Correlation coefficient value is about 0.86, which shows a strong association. So, `imdb_rating` could be a good candidate for predicting `audience_score`.

```
# Plot of Audience score Vs imdb_num_votes
ggplot(movies, aes(x = imdb_num_votes, y = audience_score)) +
  geom_point() +
  labs(title = "IMDB Number of Votes vs Audience Score", x = "IMDB Number of Votes", y = "Audience Score")
```

IMDB Number of Votes vs Audience Score



Looks like there is increase in the audience score with imdb_num_votes. Checking for the correlation coefficient:

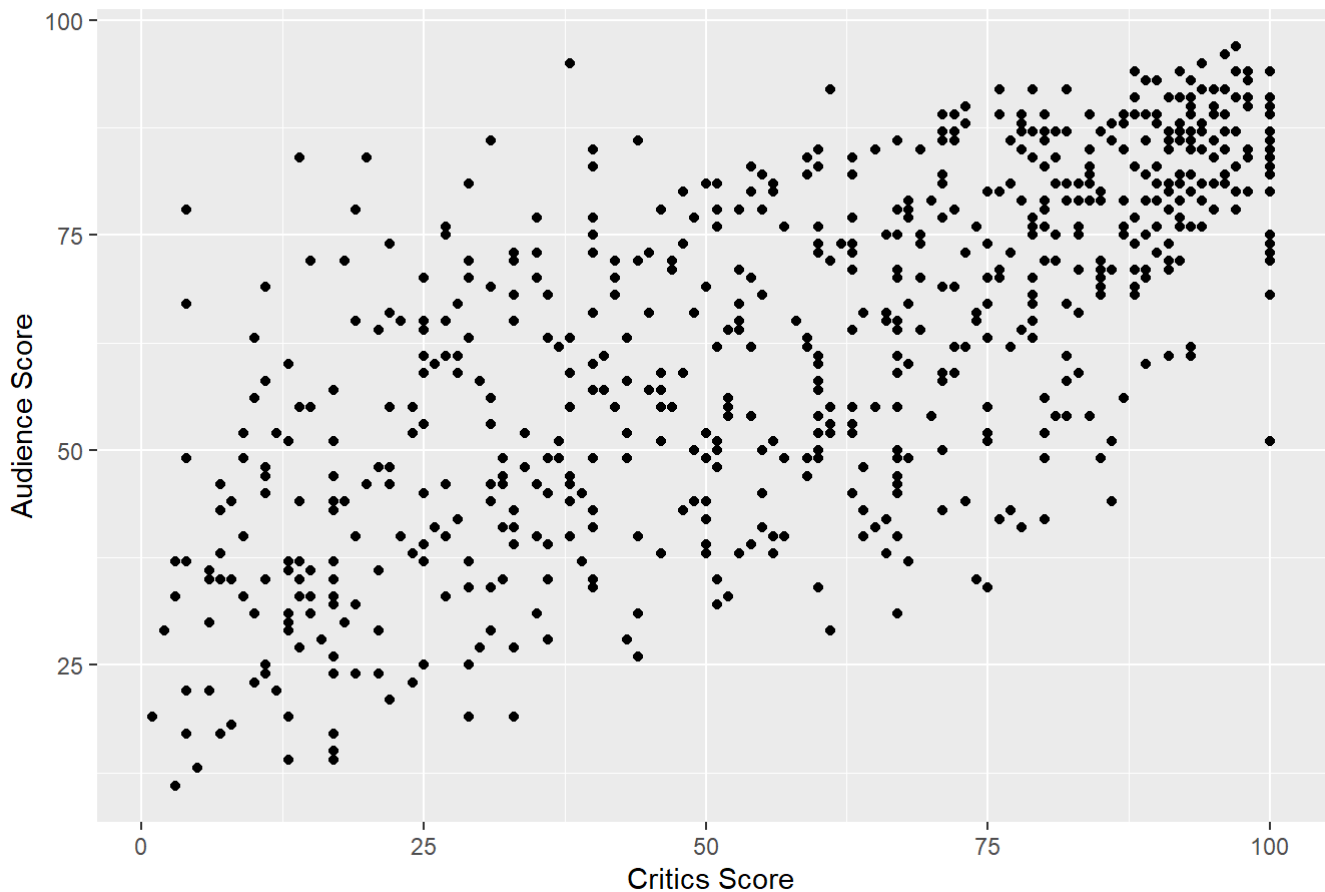
```
cor(movies$imdb_num_votes, movies$audience_score)
```

```
## [1] 0.2898128
```

Correlation coeff. is 0.29. The association is not a strong one.

```
# Plot critics_score Vs audience_score
ggplot(movies, aes(x = critics_score, y = audience_score)) +
  geom_point() +
  labs(title = "Critics Score vs Audience Score", x = "Critics Score", y = "Audience Score")
```

Critics Score vs Audience Score



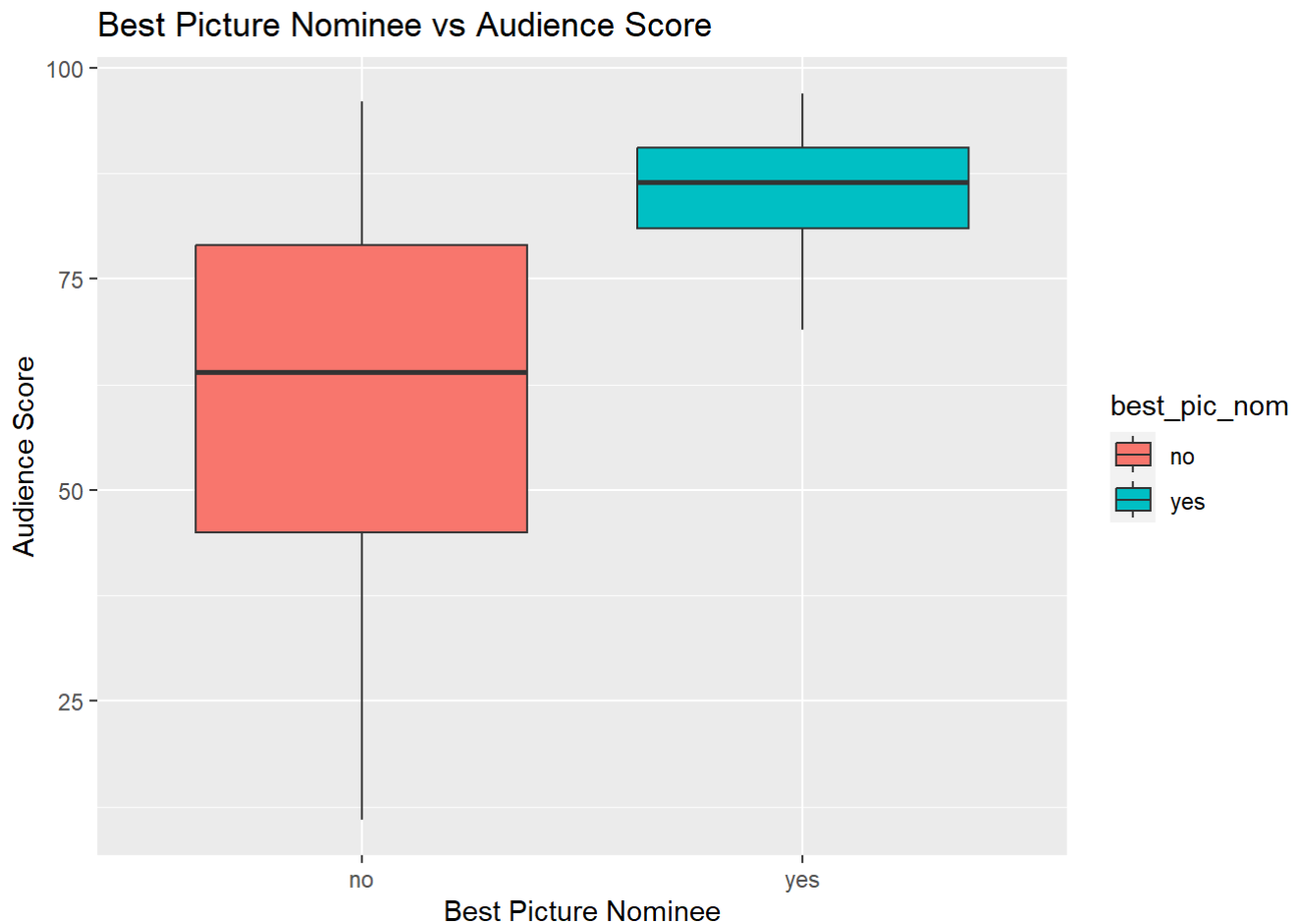
Appears to have association between critics_score and audience_score. The correlation coefficient is given by:

```
cor(movies$critics_score, movies$audience_score)
```

```
## [1] 0.7042762
```

Correlation coefficient value is about 0.70, which shows a good association. So, critics_score also could be a candidate for predicting audience_score.

```
# Plot of Audience score Vs best_pic_nom
ggplot(aes(y = audience_score, x = best_pic_nom, fill = best_pic_nom), data = movies)
+ geom_boxplot() +
  labs(title = "Best Picture Nominee vs Audience Score", x = "Best Picture Nominee",
        y = "Audience Score")
```



The audience score is high for best picture nominated movies compared to the non-nominated movies.

```
sum(movies$best_pic_nom == "yes")
```

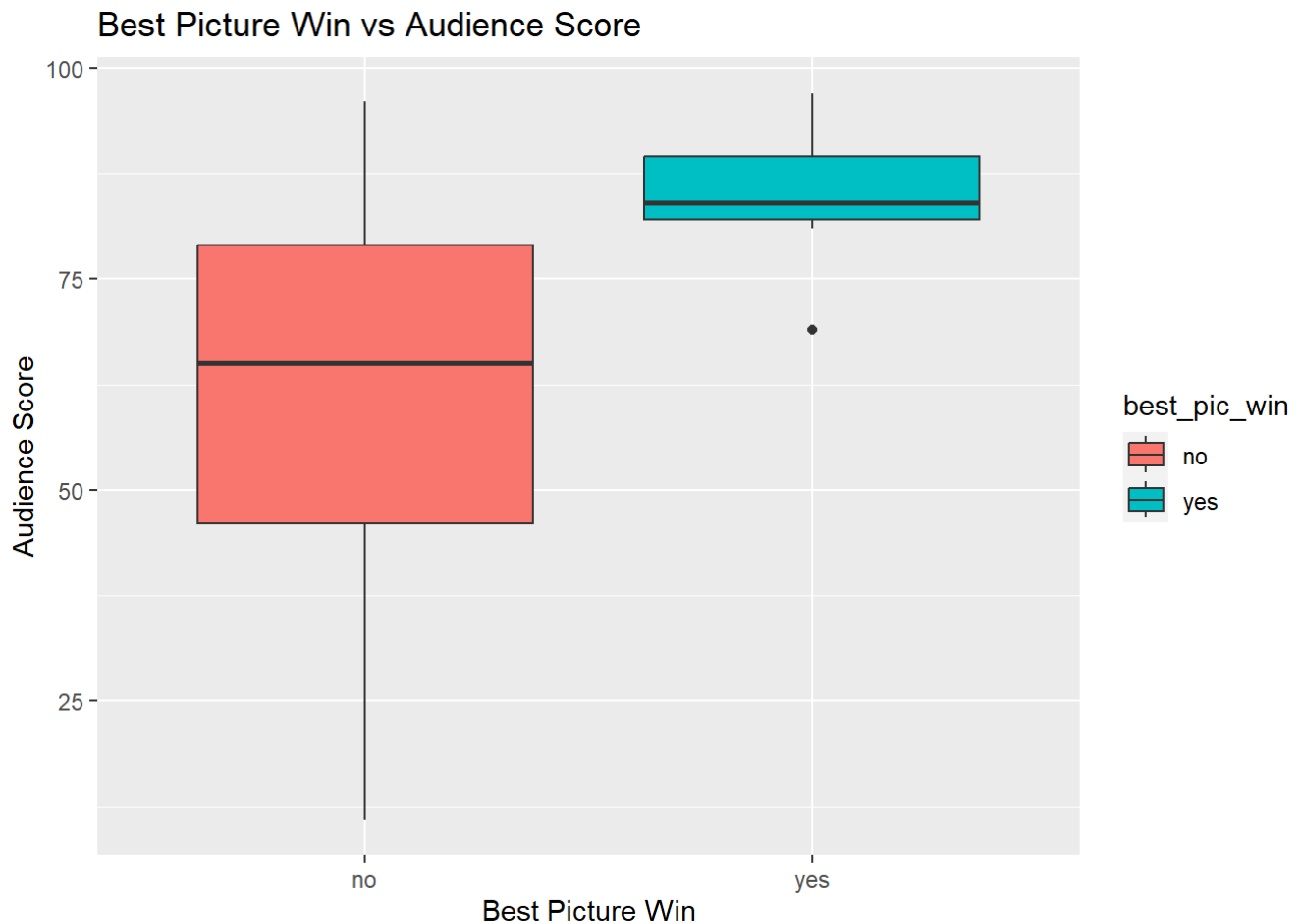
```
## [1] 22
```

```
sum(movies$best_pic_nom == "no")
```

```
## [1] 629
```

In the data, the number of nominated movies are 22 where as the non-nominated for the best picture are 629.

```
# Plot of Audience score Vs best_pic_win
ggplot(aes(y = audience_score, x = best_pic_win, fill = best_pic_win), data = movies)
+ geom_boxplot() +
  labs(title = "Best Picture Win vs Audience Score", x = "Best Picture Win", y = "Audience Score")
```



The audience score is high for movies won as best picture compared to the movies which could not win the best picture.

```
sum(movies$best_pic_win == "yes")
```

```
## [1] 7
```

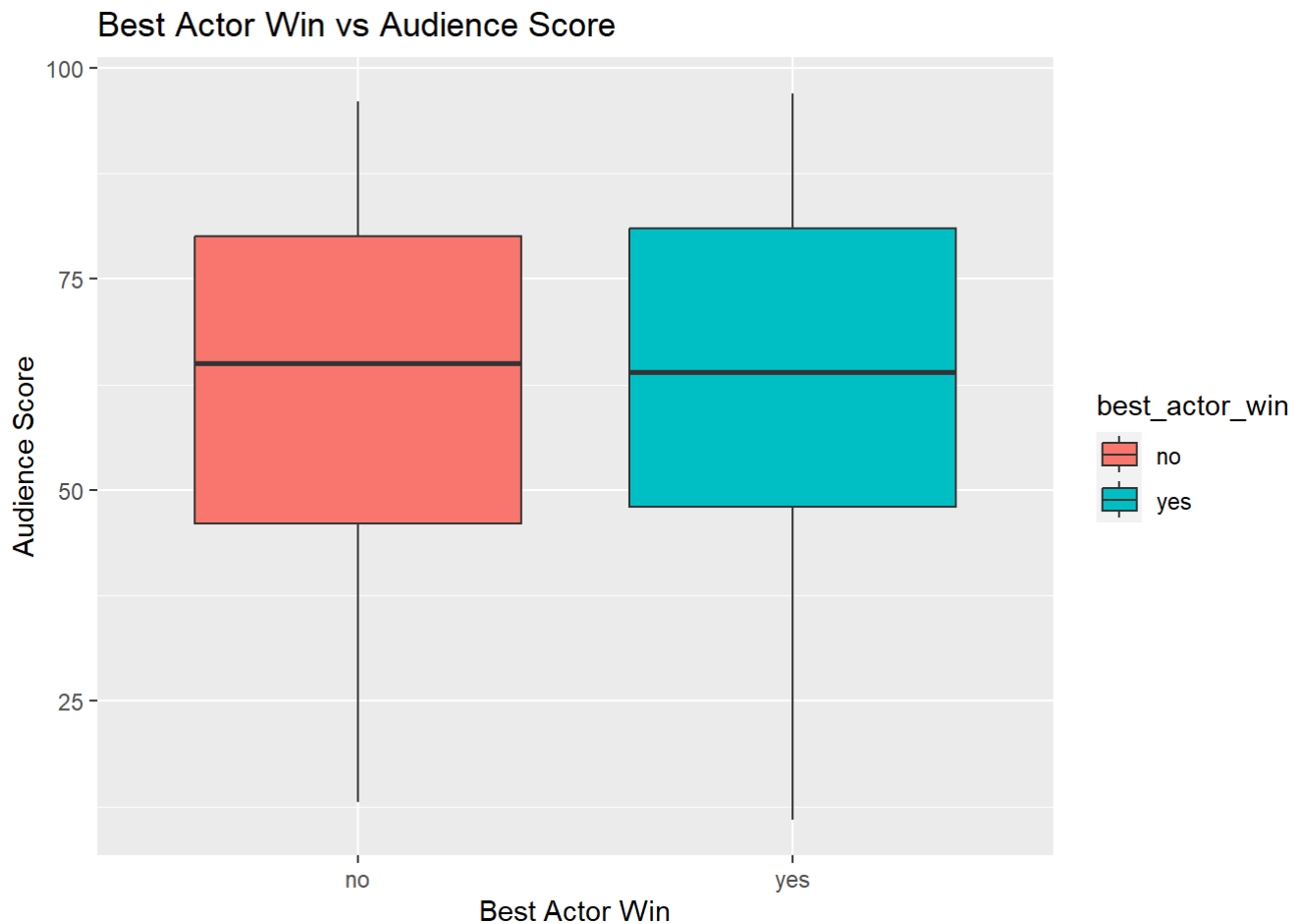
```
sum(movies$best_pic_win == "no")
```

```
## [1] 644
```

In the data, the number of best picture won are only 7 where as the which couldnot win the best picture are 644.

There is an outlier in the case of best_pic_win, but it is just one, so might not be of concern.

```
# Plot of Audience score Vs best_actor_win
ggplot(aes(y = audience_score, x = best_actor_win, fill = best_actor_win), data = movies) + geom_boxplot() +
  labs(title = "Best Actor Win vs Audience Score", x = "Best Actor Win", y = "Audience Score")
```



The audience score is almost the same for the movies best_actor_win compared to the non-best_actor_win movies.

```
sum(movies$best_actor_win == "yes")
```

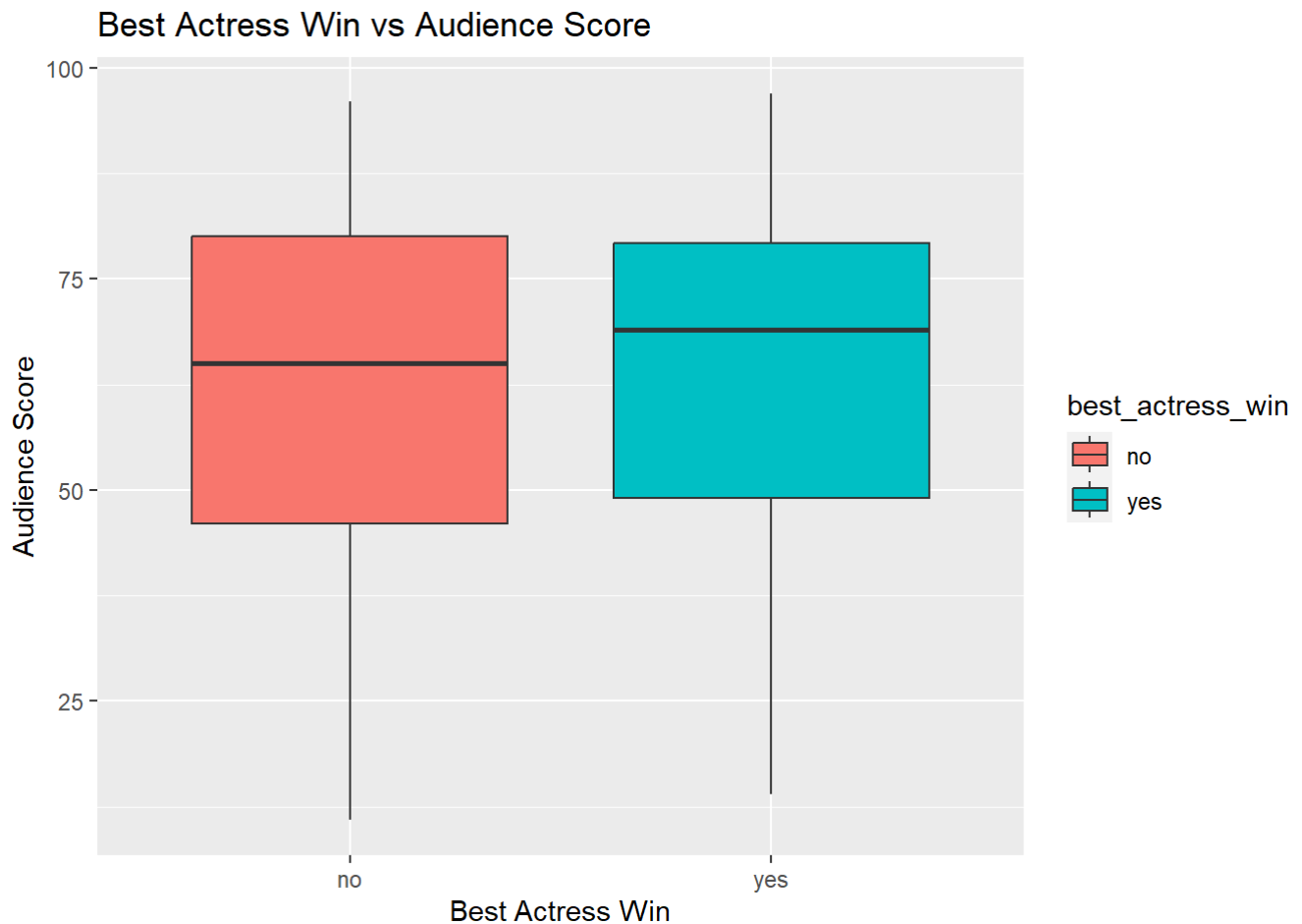
```
## [1] 93
```

```
sum(movies$best_actor_win == "no")
```

```
## [1] 558
```

In the data, the number of best_actor_win are 93 where as the ones which couldnot win the best actor are 558.

```
# Plot of Audience score Vs best_actress_win
ggplot(aes(y = audience_score, x = best_actress_win, fill = best_actress_win), data =
movies) + geom_boxplot() +
  labs(title = "Best Actress Win vs Audience Score", x = "Best Actress Win", y = "Aud
ience Score")
```



The audience score is slightly higher for the movies `best_actress_win` compared to the non-`best_actress_win` movies.

```
sum(movies$best_actress_win == "yes")
```

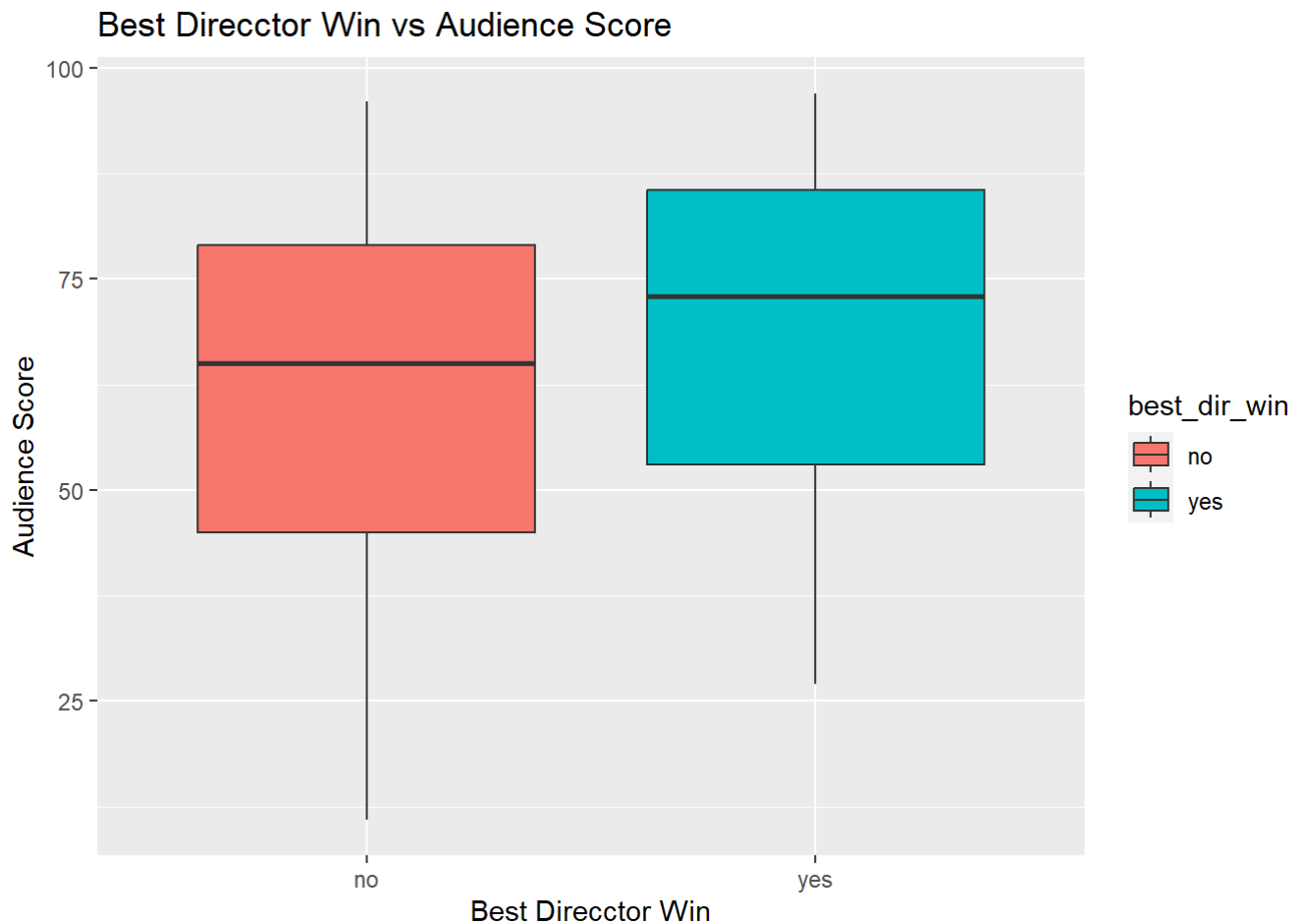
```
## [1] 72
```

```
sum(movies$best_actress_win == "no")
```

```
## [1] 579
```

In the data, the number of `best_actress_win` are 72 where as the ones which couldnot win the best actress are 579.

```
# Plot of Audience score Vs best_dir_win
ggplot(aes(y = audience_score, x = best_dir_win, fill = best_dir_win), data = movies)
+ geom_boxplot() +
  labs(title = "Best Direcctor Win vs Audience Score", x = "Best Direcctor Win", y =
"Audience Score")
```



The audience score is higher for the movies best_dir_win compared to the non-best_dir_win movies.

```
sum(movies$best_dir_win == "yes")
```

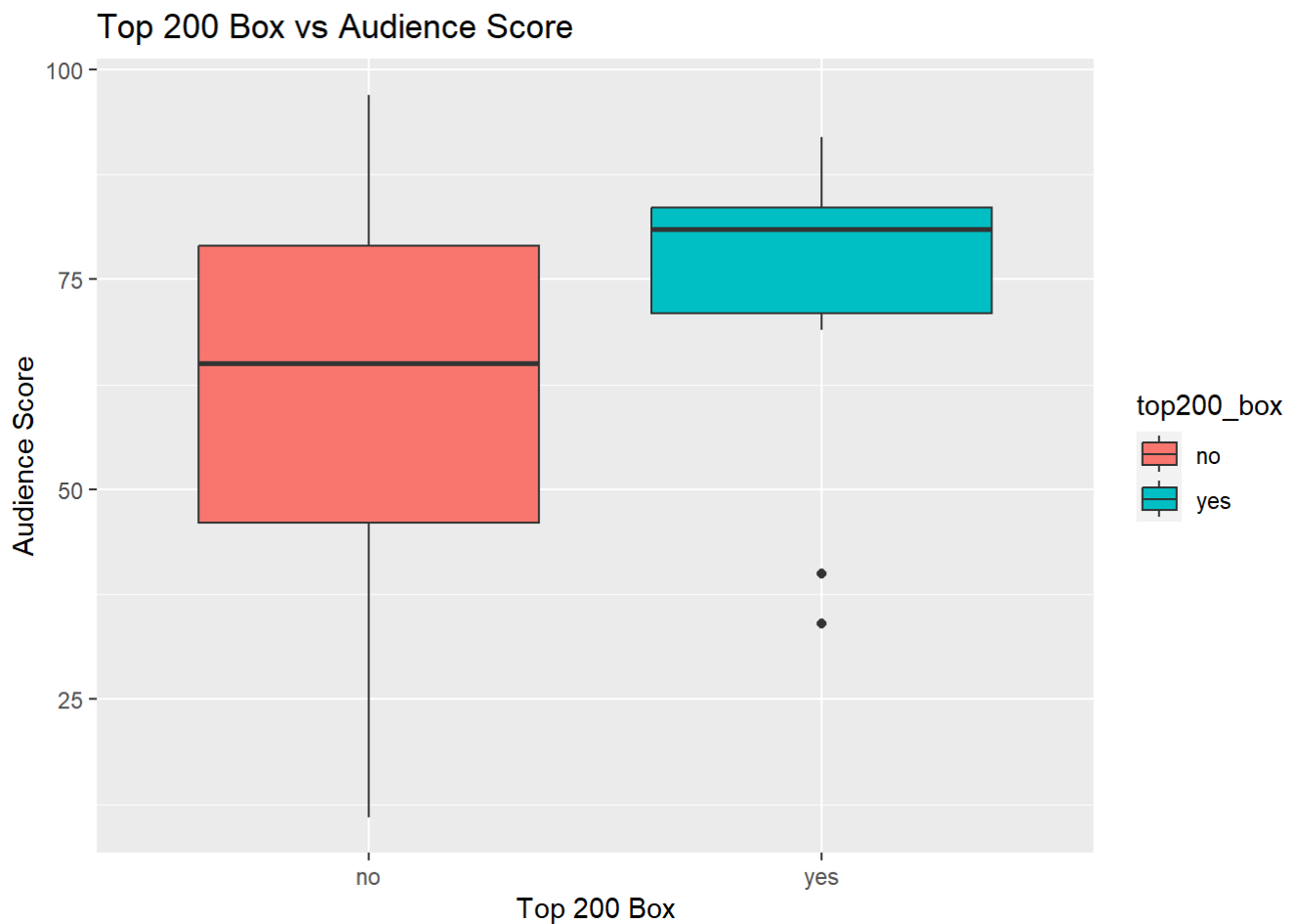
```
## [1] 43
```

```
sum(movies$best_dir_win == "no")
```

```
## [1] 608
```

In the data, the number of best_dir_win are 43 where as the ones which couldnot win the best dir are 608.

```
# Plot of Audience score Vs top200_box
ggplot(aes(y = audience_score, x = top200_box, fill = top200_box), data = movies) + g
eom_boxplot() +
  labs(title = "Top 200 Box vs Audience Score", x = "Top 200 Box", y = "Audience Scor
e")
```



The audience score is higher for the movies top200_box compared to the non-top200_box movies.

```
sum(movies$top200_box == "yes")
```

```
## [1] 15
```

```
sum(movies$top200_box == "no")
```

```
## [1] 636
```

In the data, the number of top200_box are 15 where as the ones which could not be in the top200_box are 636. It can be seen that there are a couple of outliers in the case of top200_box. Not sure whether this is a matter of concern. Perhaps we need to see more data to conclude on this.

Data Pre-processing

Extract the above columns data into a data frame, so that it can be used for regression.

```
moviesReg <- movies[c('audience_score', 'feature_film', 'drama', 'runtime', 'mpaa_rating_R', 'thtr_rel_year', 'oscar_season', 'summer_season', 'imdb_rating', 'imdb_num_votes', 'critics_score', 'best_pic_nom', 'best_pic_win', 'best_actor_win', 'best_actress_win', 'best_dir_win', 'top200_box')]
```

```
# Omitting NAs
moviesReg <- na.omit(moviesReg)
```

Summary

Summary of the various parameters is given below:

```
summary(moviesReg)
```

```
## audience_score feature_film drama runtime mpaa_rating_R
## Min. :11.00 no : 59 no :345 Min. : 39.0 no :321
## 1st Qu.:46.00 yes:591 yes:305 1st Qu.: 92.0 yes:329
## Median :65.00 Median :103.0
## Mean :62.35 Mean :105.8
## 3rd Qu.:80.00 3rd Qu.:115.8
## Max. :97.00 Max. :267.0
## thtr_rel_year oscar_season summer_season imdb_rating imdb_num_votes
## Min. :1970 no :460 no :442 Min. :1.900 Min. : 180
## 1st Qu.:1990 yes:190 yes:208 1st Qu.:5.900 1st Qu.: 4584
## Median :2000 Median :6.600 Median : 15204
## Mean :1998 Mean :6.492 Mean : 57620
## 3rd Qu.:2007 3rd Qu.:7.300 3rd Qu.: 58484
## Max. :2014 Max. :9.000 Max. :893008
## critics_score best_pic_nom best_pic_win best_actor_win best_actress_win
## Min. : 1.00 no :628 no :643 no :557 no :578
## 1st Qu.: 33.00 yes: 22 yes: 7 yes: 93 yes: 72
## Median : 61.00
## Mean : 57.65
## 3rd Qu.: 83.00
## Max. :100.00
## best_dir_win top200_box
## no :607 no :635
## yes: 43 yes: 15
##
##
##
##
```

4. Modeling

Developing a Bayesian regression model to predict `audience_score` from the following explanatory variables: `feature_film`, `drama`, `runtime`, `mpaa_rating_R`, `thtr_rel_year`, `oscar_season`, `summer_season`, `imdb_rating`, `imdb_num_votes`, `critics_score`, `best_pic_nom`, `best_pic_win`, `best_actor_win`, `best_actress_win`, `best_dir_win`, `top200_box`. Different models are discussed with reference to the given data. From these, finally, we select a model.

Reference: Essentially the methods described in chapters 7 and 8 of “An Introduction to Bayesian Thinking” will be followed.

Bayesian Models and Diagnostics using method = “MCMC”

We run `bas.lm` function from the BAS package. We first run the full model and use this information for later decision on what variables to include. Here we have 16 potential predictors. The total number of models is $2^{16} = 65536$. We want to illustrate how to explore models using stochastic methods. Hence we set argument `method = MCMC` inside the `bas.lm` function. We also use the Zellner-Siow cauchy prior for the prior distributions of the coefficients in this regression.

```
## Load the MASS library
library(MASS)
```

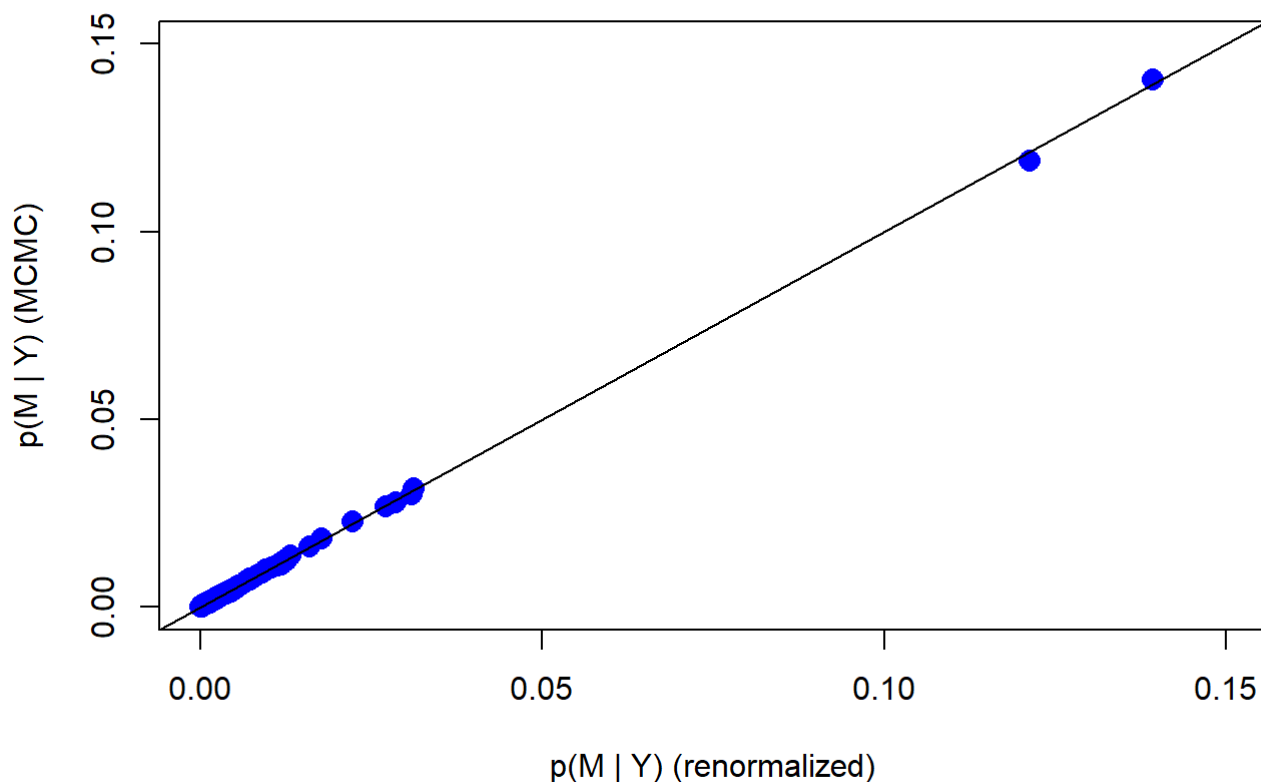
```
movies.ZS.mcmc = bas.lm(audience_score ~ ., data=moviesReg, prior="ZS-null", modelprior=uniform(), method = "MCMC")
```

Diagnostic Plots

To analyze the result, we first look at the diagnostic plot using `diagnostics` function and see whether we have run the MCMC exploration long enough so that the posterior inclusion probability (pip) has converged.

```
diagnostics(movies.ZS.mcmc, type = "model", col = "blue", pch = 16, cex = 1.5)
```

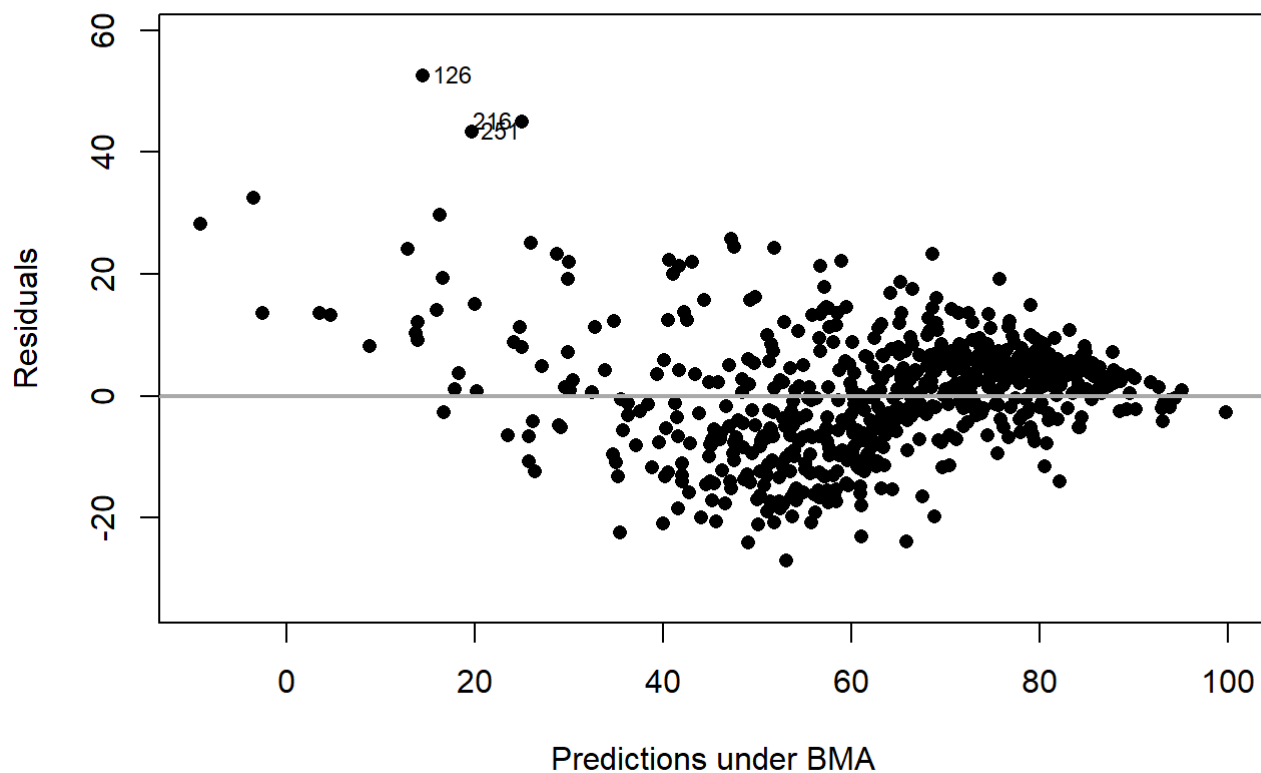
Convergence Plot: Posterior Model Probabilities



we see that most points stay in the 45 degree diagonal line, meaning the posterior inclusion probability from the MCMC method has mostly converged to the theoretical posterior inclusion probability.

1. Residuals Versus Fitted Values Using BMA

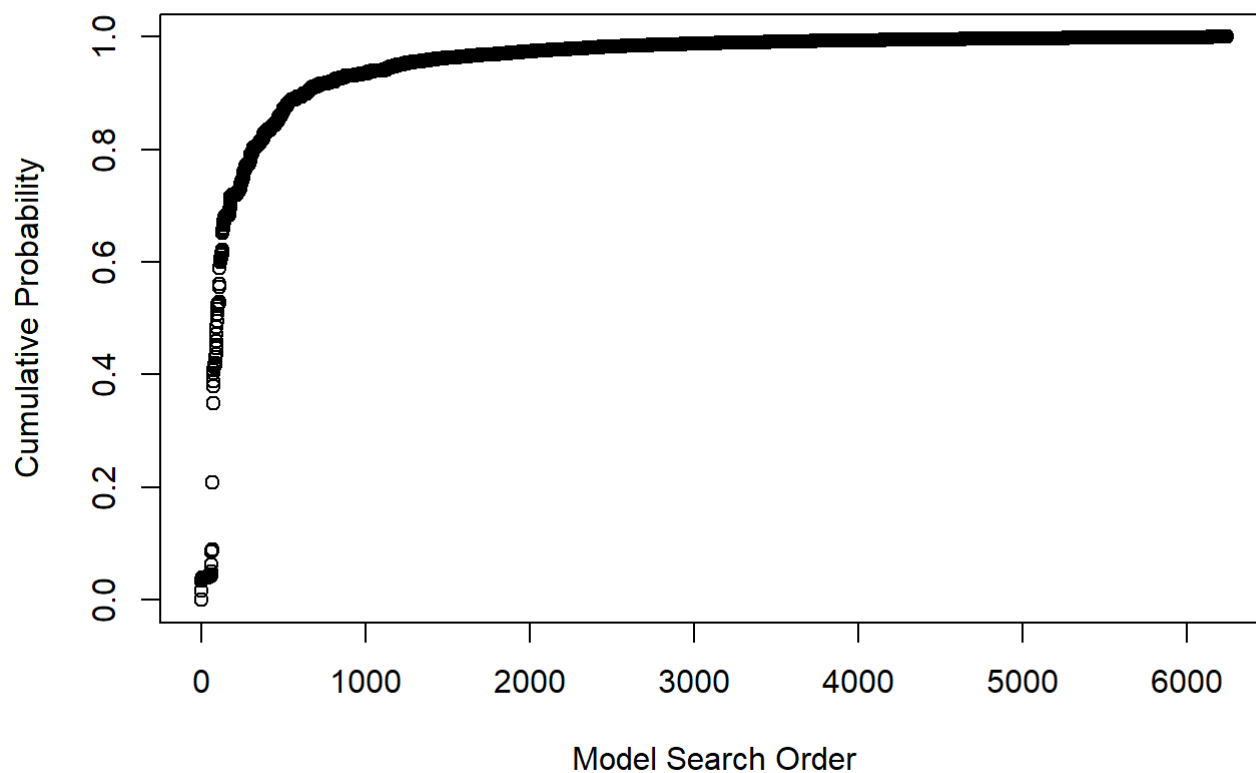
```
plot(movies.ZS.mcmc, which = 1, add.smooth = F, ask = F, pch = 16, sub.caption="", caption="")
abline(a = 0, b = 0, col = "darkgrey", lwd = 2)
```



The residuals lie around the dash line $= 0$, and has a constant variance. Observations 126, 216, and 251 may be the potential outliers, which are indicated in the plot.

2. Cumulative Sampled Probability

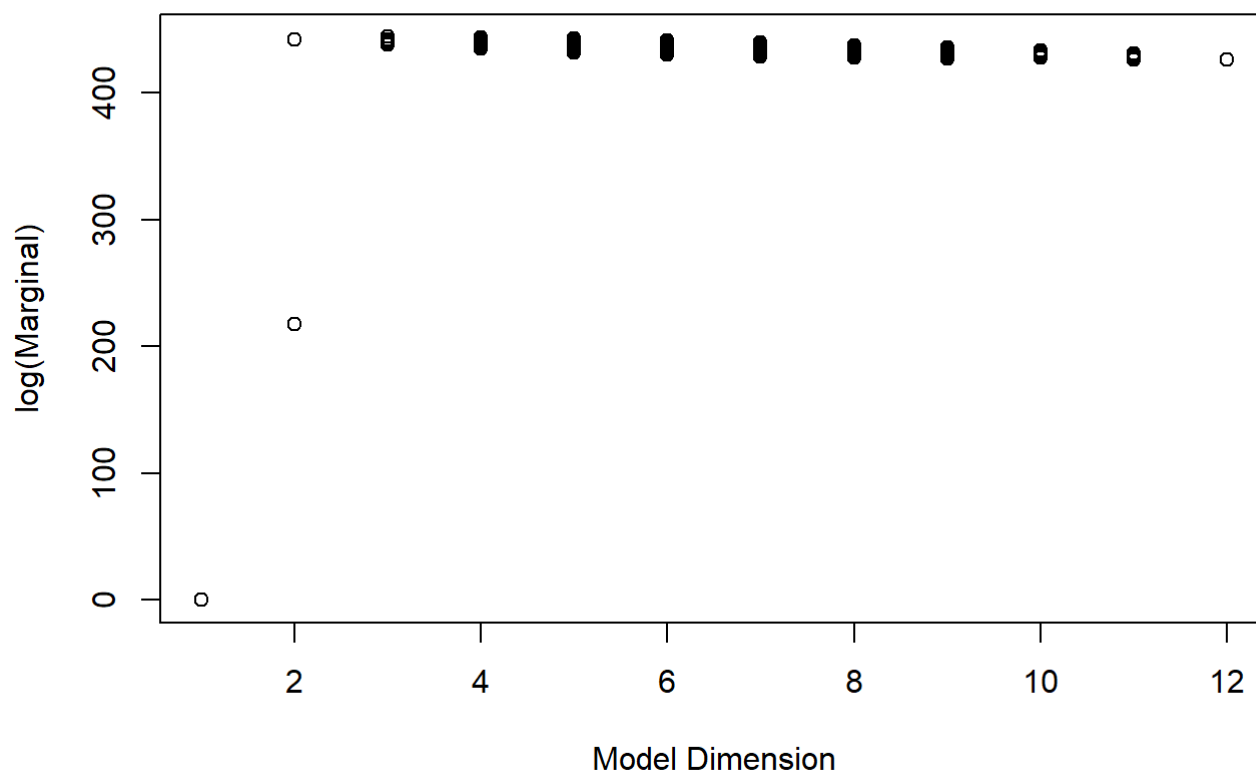
```
plot(movies.ZS.mcmc, which=2, add.smooth = F, sub.caption="", caption="")
```



After 4,000 unique models with MCMC sampling, the probability is starting to level off, indicating that these additional models have very small probability and do not contribute substantially to the posterior distribution.

3. Model Complexity

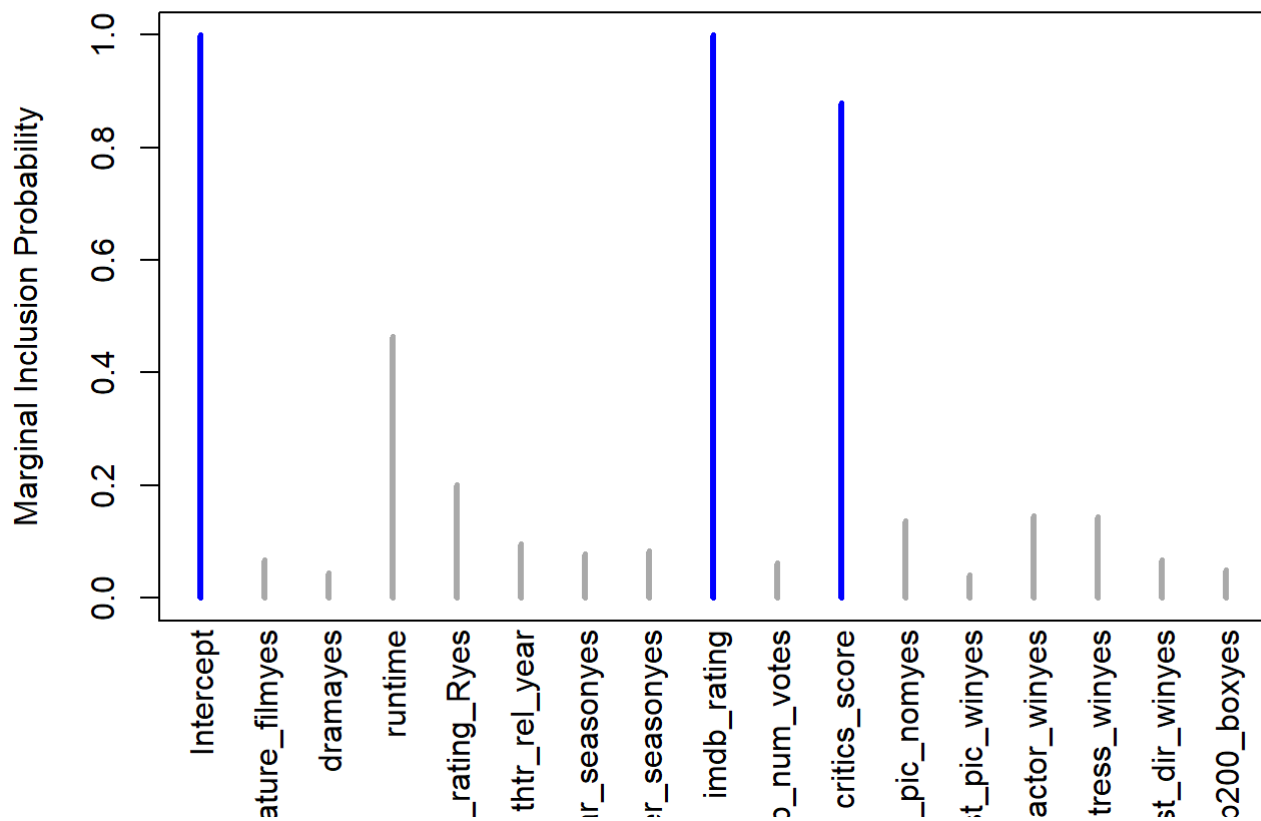
```
plot(movies.ZS.mcmc, which=3, ask=F, caption="", sub.caption="")
```



The graph doesn't show any models with the highest Bayes factors or logs of marginal likelihoods (perhaps more study needs to be done).

4. Marginal Inclusion Probability Below is the plot showing the importance of different predictors

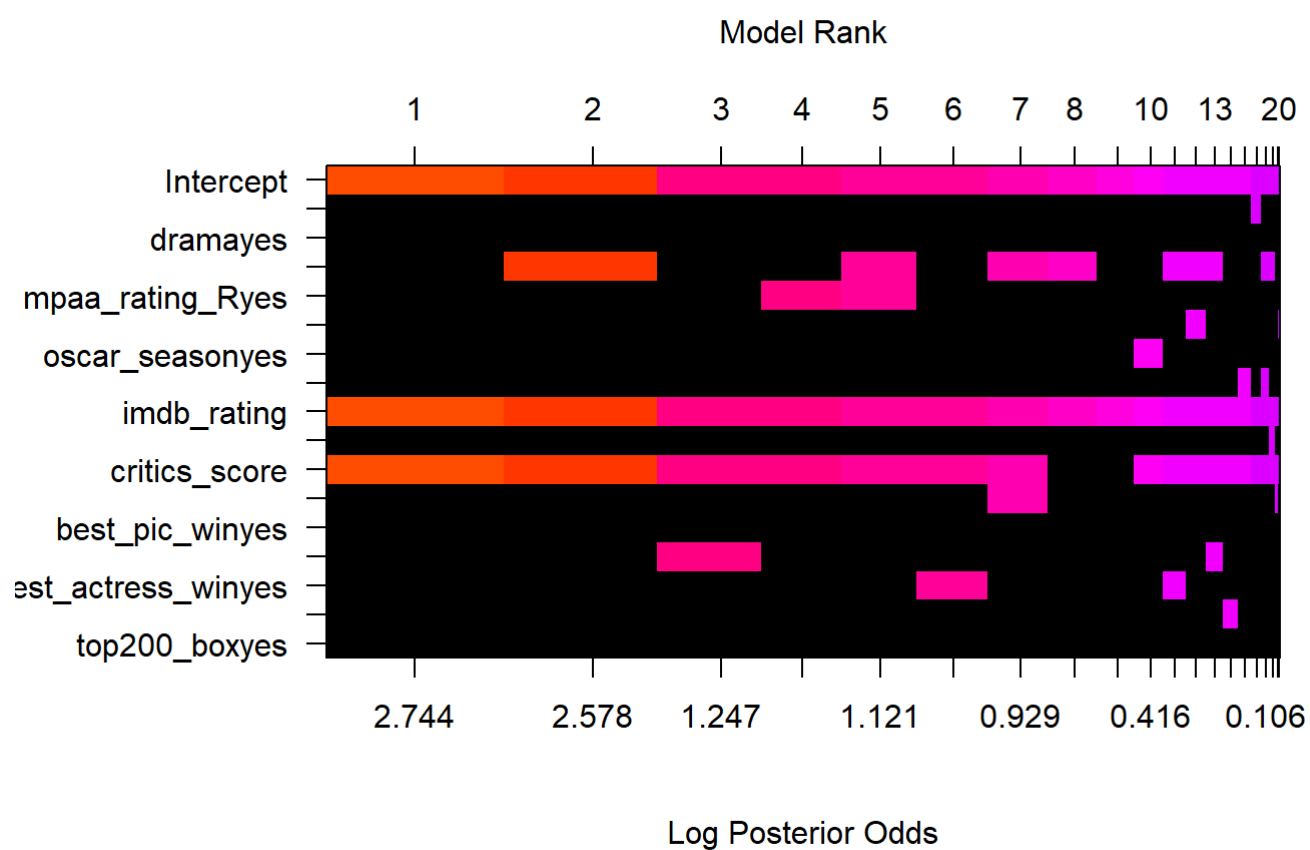
```
plot(movies.ZS.mcmc, which = 4, ask = F, caption = "", sub.caption = "", col.in = "blue", col.ex = "darkgrey", lwd = 3)
```



The lines in blue correspond to the variables where the marginal posterior inclusion probability (pip), is greater than 0.5, suggesting that these variables are important for prediction. The variables represented in grey lines have posterior inclusion probability less than 0.5.

5. Model Space Visualization To focus on the high posterior probability models, we can look at the image of the model space.

```
image(movies.ZS.mcmc, rotate = F)
```



imdb_rating and critics_score are included in top 7 models.

The coefficients and credible intervals of each variable are given below

```
# Extract coefficients
coef.movies.ZS.mcmc = coef(movies.ZS.mcmc)
coef.movies.ZS.mcmc
```

```
##
## Marginal Posterior Summaries of Coefficients:
##
## Using BMA
##
## Based on the top 6244 models
##
```

	post mean	post SD	post p(B != 0)
## Intercept	6.235e+01	3.946e-01	1.000e+00
## feature_filmyes	-1.095e-01	5.773e-01	6.850e-02
## dramayes	1.777e-02	2.006e-01	4.547e-02
## runtime	-2.537e-02	3.112e-02	4.644e-01
## mpaa_rating_Ryes	-3.053e-01	7.040e-01	2.014e-01
## thtr_rel_year	-4.825e-03	1.875e-02	9.598e-02
## oscar_seasonyes	-8.303e-02	3.831e-01	7.831e-02
## summer_seasonyes	8.999e-02	3.890e-01	8.353e-02
## imdb_rating	1.497e+01	7.381e-01	1.000e+00
## imdb_num_votes	2.292e-07	1.376e-06	6.216e-02
## critics_score	6.223e-02	3.071e-02	8.802e-01
## best_pic_nomyes	5.369e-01	1.612e+00	1.381e-01
## best_pic_winyes	-1.081e-02	8.669e-01	4.117e-02
## best_actor_winyes	-2.909e-01	8.344e-01	1.469e-01
## best_actress_winyes	-3.141e-01	9.113e-01	1.443e-01
## best_dir_winyes	-1.195e-01	6.224e-01	6.773e-02
## top200_boxyes	9.110e-02	7.229e-01	5.010e-02

```
round(confint(coef.movies.ZS.mcmc), 4)
```

```
##
```

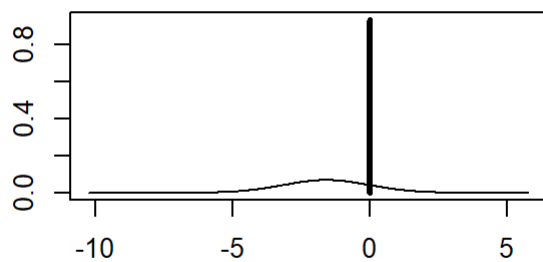
	2.5%	97.5%	beta
## Intercept	61.5717	63.0954	62.3477
## feature_filmyes	-1.0385	0.0000	-0.1095
## dramayes	0.0000	0.0000	0.0178
## runtime	-0.0829	0.0000	-0.0254
## mpaa_rating_Ryes	-2.0863	0.0006	-0.3053
## thtr_rel_year	-0.0557	0.0009	-0.0048
## oscar_seasonyes	-0.9638	0.0000	-0.0830
## summer_seasonyes	-0.0152	1.0671	0.0900
## imdb_rating	13.6394	16.5483	14.9652
## imdb_num_votes	0.0000	0.0000	0.0000
## critics_score	0.0000	0.1044	0.0622
## best_pic_nomyes	0.0000	5.0821	0.5369
## best_pic_winyes	0.0000	0.0000	-0.0108
## best_actor_winyes	-2.6575	0.0000	-0.2909
## best_actress_winyes	-2.8397	0.0000	-0.3141
## best_dir_winyes	-1.4754	0.0040	-0.1195
## top200_boxyes	0.0000	0.0000	0.0911

```
## attr(,"Probability")
## [1] 0.95
## attr(,"class")
## [1] "confint.bas"
```

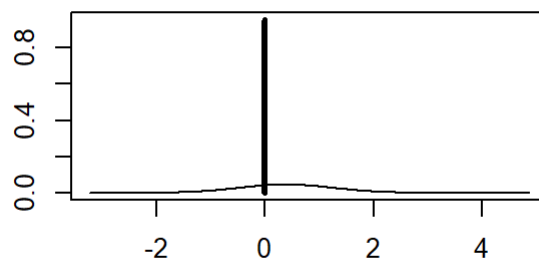
We can also plot the posterior distributions of these coefficients to take a closer look at the distributions

```
par(mfrow = c(2, 2))  
plot(coef.movies.ZS.mcmc, subset = c(2:17))
```

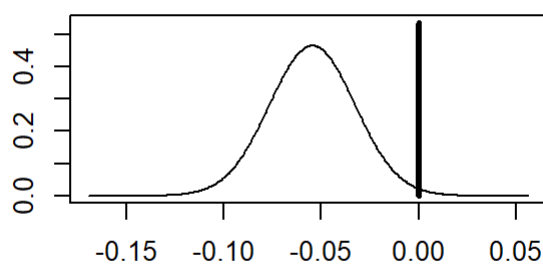
feature_filmyes



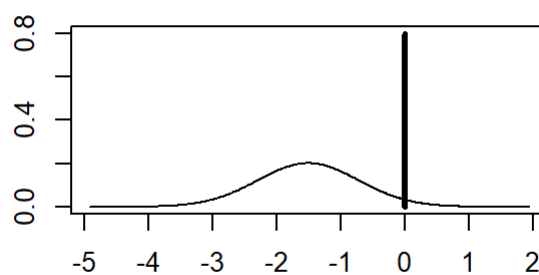
dramayes



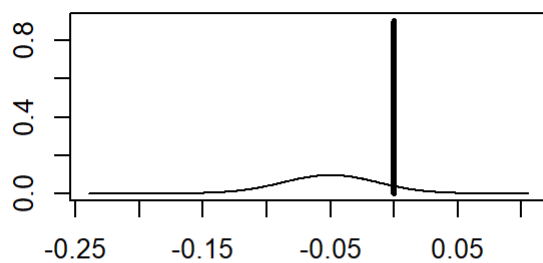
runtime



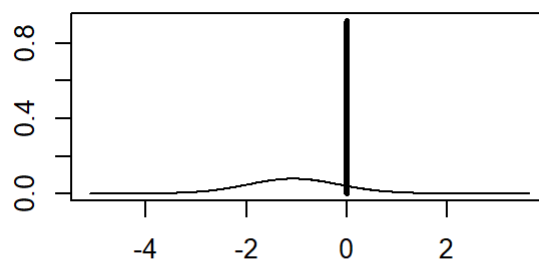
mpaa_rating_Ryes



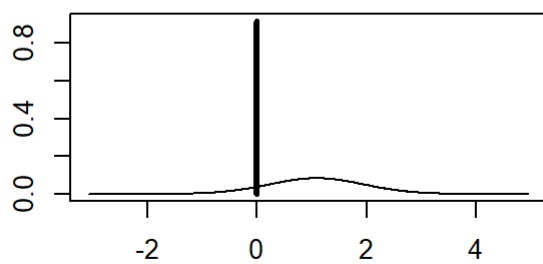
thtr_rel_year



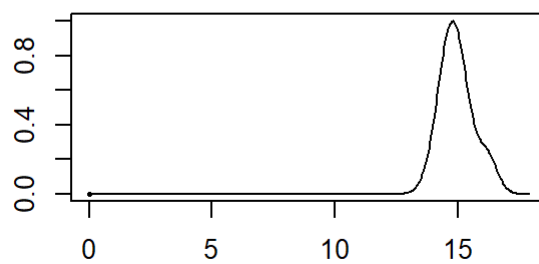
oscar_seasonyes



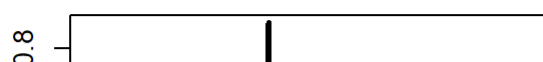
summer_seasonyes



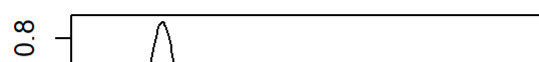
imdb_rating

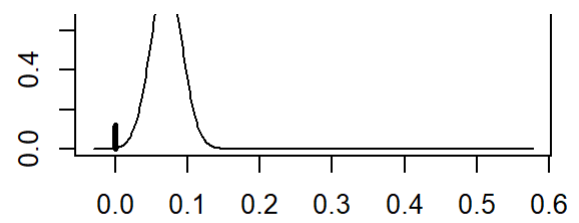
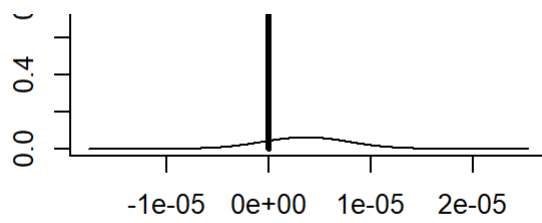


imdb_num_votes



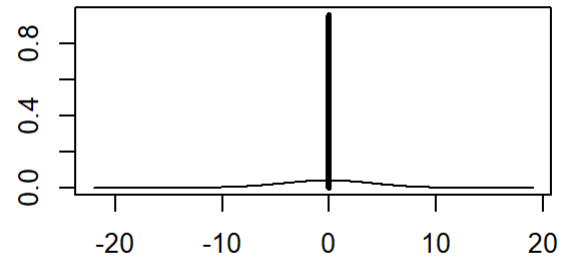
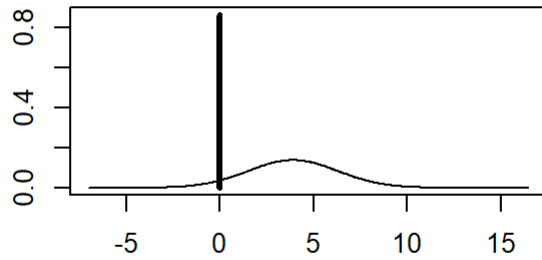
critics_score





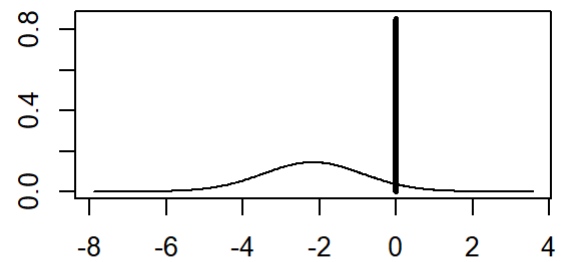
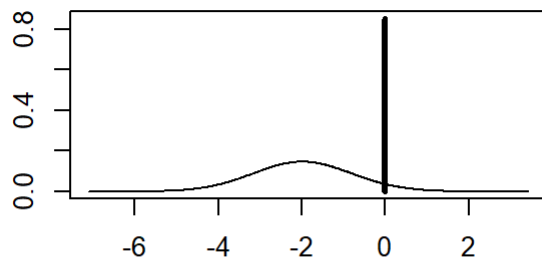
best_pic_nomyes

best_pic_winyes



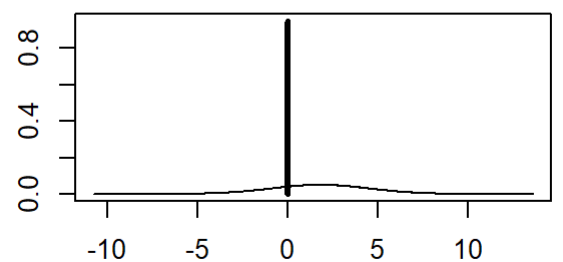
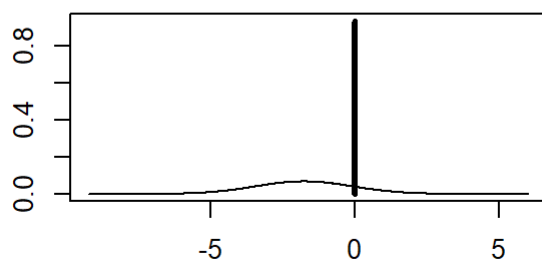
best_actor_winyes

best_actress_winyes



best_dir_winyes

top200_boxyes



This plot agrees with the summary table we obtained above, which shows that the posterior probability distributions of most of the predictors have a very large point mass at 0, while the distribution of critics_score and imdb_rating have a relatively small mass at 0. There is a slightly little tip at 0 for the variable imdb_rating, indicating that the posterior inclusion probability of imdb_rating is not exactly 1. However, since the probability mass for imdb_rating to be 0 is so small, that we are almost certain that imdb_rating should be included under Bayesian model averaging.

Decision Making Under Model Uncertainty

we will talk about different methods for selecting models:

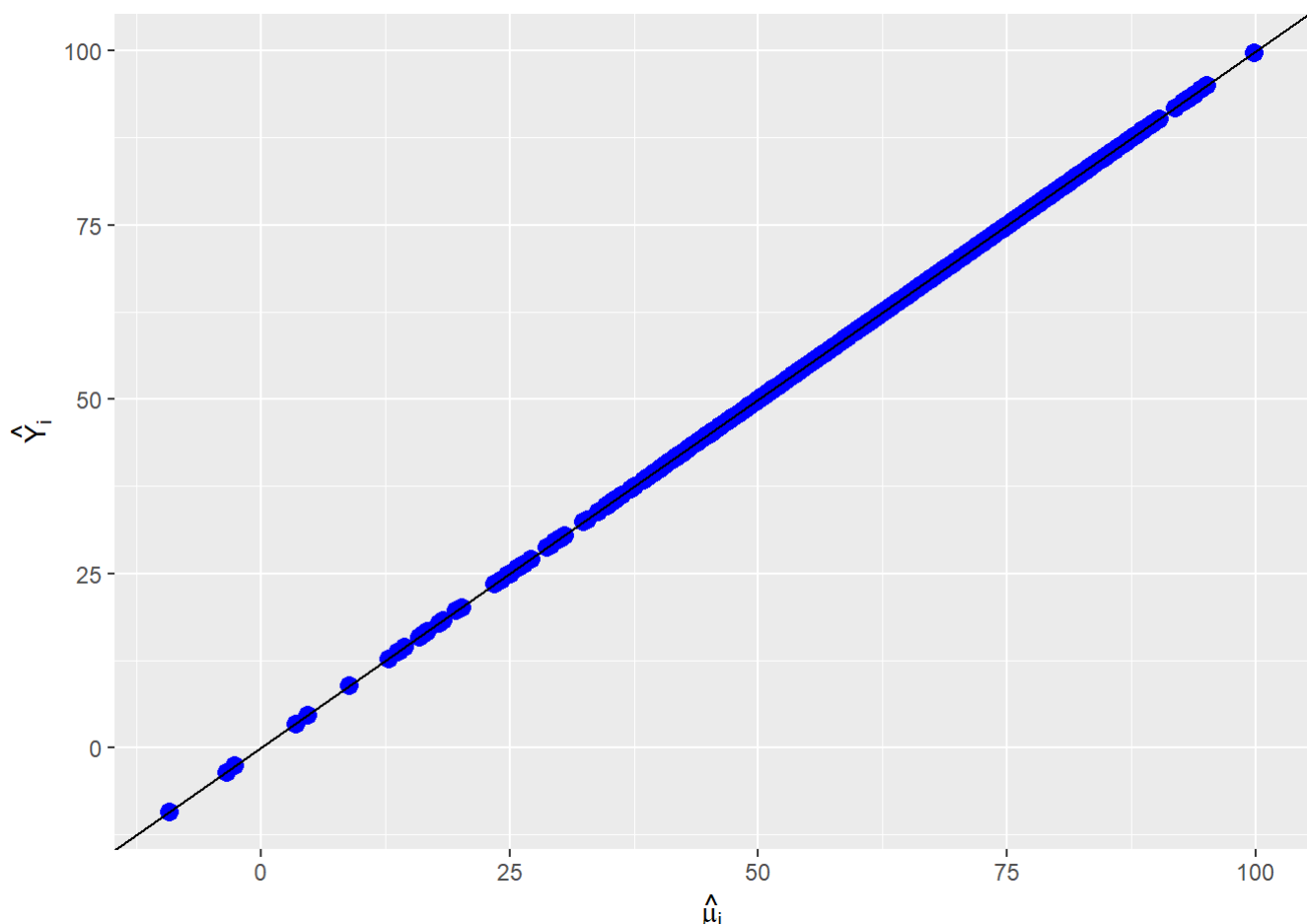
1. Bayesian Model Averaging (BMA): BMA provides a weighted average of models based on their posterior probabilities. This can improve predictive performance and robustness by incorporating model uncertainty.
2. Highest Probability Model: Given the data, this model is having the highest posterior probability.
3. Median Probability Model: This model includes variables with a posterior probability of inclusion greater than 0.5.
4. Best Predictive Model: This model is having the best predictive performance.

Each of these models has its strengths and is suited to different scenarios. If your focus is on interpretability and identifying key variables, the Highest Probability or Median Probability models might be more appropriate. If prediction accuracy is your main concern, the Best Predictive Model or Bayesian Model Averaging could be better choices.

BMA Model

```
movies.BMA = predict(movies.ZS.mcmc, estimator = "BMA")
mu_hat = fitted(movies.ZS.mcmc, estimator = "BMA")

library(ggplot2)
output = data.frame(mu_hat = mu_hat, fitted = movies.BMA$fit)
# Plot result from `fitted` function and result from `fit` attribute
ggplot(data = output, aes(x = mu_hat, y = fitted)) +
  geom_point(pch = 16, color = "blue", size = 3) +
  geom_abline(intercept = 0, slope = 1) +
  xlab(expression(hat(mu)[i])) + ylab(expression(hat(Y[i])))
```



Plotting the two sets of fitted values, one obtained from the fitted function, another obtained from the fit attribute of the predict object movies.BMA, we see that they are in perfect agreement.

```
# Posterior means of Coefficients
coef(movies.ZS.mcmc, estimator="BMA")
```

```
##
## Marginal Posterior Summaries of Coefficients:
##
## Using BMA
##
## Based on the top 6244 models
##
```

	post mean	post SD	post p(B != 0)
## Intercept	6.235e+01	3.946e-01	1.000e+00
## feature_filmyes	-1.095e-01	5.773e-01	6.850e-02
## dramayes	1.777e-02	2.006e-01	4.547e-02
## runtime	-2.537e-02	3.112e-02	4.644e-01
## mpaa_rating_Ryes	-3.053e-01	7.040e-01	2.014e-01
## thtr_rel_year	-4.825e-03	1.875e-02	9.598e-02
## oscar_seasonyes	-8.303e-02	3.831e-01	7.831e-02
## summer_seasonyes	8.999e-02	3.890e-01	8.353e-02
## imdb_rating	1.497e+01	7.381e-01	1.000e+00
## imdb_num_votes	2.292e-07	1.376e-06	6.216e-02
## critics_score	6.223e-02	3.071e-02	8.802e-01
## best_pic_nomyes	5.369e-01	1.612e+00	1.381e-01
## best_pic_winyes	-1.081e-02	8.669e-01	4.117e-02
## best_actor_winyes	-2.909e-01	8.344e-01	1.469e-01
## best_actress_winyes	-3.141e-01	9.113e-01	1.443e-01
## best_dir_winyes	-1.195e-01	6.224e-01	6.773e-02
## top200_boxyes	9.110e-02	7.229e-01	5.010e-02

Under Bayesian model averaging, the table above provides the posterior mean, the posterior standard deviation, and the posterior inclusion probability (pip) of each coefficient. The posterior mean of the coefficient under BMA would be used for future predictions. The posterior standard deviation provides measure of variability of the coefficient.

Under BMA, `imdb_rating` has posterior inclusion probability 1, suggesting that it is very likely that `imdb_rating` should be included in the model. `critics_score` also has a high posterior inclusion probability of about 0.88, followed by `runtime` with 0.46. However, the posterior inclusion probability of other variables are relatively small compared to `critics_score`.

Highest Probability Model

```
movies.HPM = predict(movies.ZS.mcmc, estimator = "HPM")

# Extract the variable names
movies.HPM$best.var
```

```
## [1] "Intercept"      "imdb_rating"    "critics_score"
```

we see that only `imdb_rating`, and `critics_score` are included in the model

```
# Posterior means of coefficients
coef(movies.ZS.mcmc, estimator="HPM")
```

```
##
## Marginal Posterior Summaries of Coefficients:
##
## Using HPM
##
## Based on the top 1 models
##
```

	post mean	post SD	post p(B != 0)
## Intercept	62.34769	0.39549	1.00000
## feature_filmyes	0.00000	0.00000	0.06850
## dramayes	0.00000	0.00000	0.04547
## runtime	0.00000	0.00000	0.46444
## mpaa_rating_Ryes	0.00000	0.00000	0.20144
## thtr_rel_year	0.00000	0.00000	0.09598
## oscar_seasonyes	0.00000	0.00000	0.07831
## summer_seasonyes	0.00000	0.00000	0.08353
## imdb_rating	14.64833	0.56593	1.00000
## imdb_num_votes	0.00000	0.00000	0.06216
## critics_score	0.07316	0.02161	0.88018
## best_pic_nomyes	0.00000	0.00000	0.13805
## best_pic_winyes	0.00000	0.00000	0.04117
## best_actor_winyes	0.00000	0.00000	0.14694
## best_actress_winyes	0.00000	0.00000	0.14428
## best_dir_winyes	0.00000	0.00000	0.06773
## top200_boxyes	0.00000	0.00000	0.05010

```
postprob.HPM = movies.ZS.mcmc$postprobs[movies.HPM$best]
postprob.HPM
```

```
## [1] 0.1404395
```

we see that this highest probability model has posterior probability of only 0.1393729. There are many models that have comparable posterior probabilities. So even this model has the highest posterior probability, we are still pretty unsure about whether it is the best model.

Median Probability Model

Another model that is frequently reported, is the median probability model (MPM). This model includes all predictors whose marginal posterior inclusion probabilities are greater than 0.5.

```
movies.MPM = predict(movies.ZS.mcmc, estimator = "MPM")
movies.MPM$best.vars
```

```
## [1] "Intercept"      "imdb_rating"    "critics_score"
```

In this “MPM” model also only `imdb_rating`, and `critics_score` are included in the model

```
# Obtain coefficients of the Median Probabilty Model
coef(movies.ZS.mcmc, estimator = "MPM")
```

```
##
## Marginal Posterior Summaries of Coefficients:
##
## Using MPM
##
## Based on the top 1 models
##
##           post mean  post SD  post p(B != 0)
## Intercept          62.34769    0.39549    1.00000
## feature_filmyes      0.00000    0.00000    0.00000
## dramayes             0.00000    0.00000    0.00000
## runtime              0.00000    0.00000    0.00000
## mpaa_rating_Ryes     0.00000    0.00000    0.00000
## thtr_rel_year        0.00000    0.00000    0.00000
## oscar_seasonyes      0.00000    0.00000    0.00000
## summer_seasonyes     0.00000    0.00000    0.00000
## imdb_rating          14.64833    0.56593    1.00000
## imdb_num_votes       0.00000    0.00000    0.00000
## critics_score        0.07316    0.02161    1.00000
## best_pic_nomyes      0.00000    0.00000    0.00000
## best_pic_winyes      0.00000    0.00000    0.00000
## best_actor_winyes    0.00000    0.00000    0.00000
## best_actress_winyes  0.00000    0.00000    0.00000
## best_dir_winyes      0.00000    0.00000    0.00000
## top200_boxyes        0.00000    0.00000    0.00000
```

Best Predictive Model

Using the squared error loss, the best predictive model is the one whose predictions are closest to BMA.

```
movies.BPM = predict(movies.ZS.mcmc, estimator = "BPM")
movies.BPM$best.vars
```

```
## [1] "Intercept"      "runtime"         "imdb_rating"     "critics_score"
```

This model includes runtime in addition to imdb_rating, and critics_score

Using the `se.fit = TRUE` option with `predict` we can calculate standard deviations for the predictions or for the mean. Then we can use this as input for the `confint` function for the prediction object

```
movies.BPM = predict(movies.ZS.mcmc, estimator = "BPM", se.fit = TRUE)
movies.BPM.conf.fit = confint(movies.BPM, parm = "mean")
movies.BPM.conf.pred = confint(movies.BPM, parm = "pred")
head(cbind(movies.BPM$fit, movies.BPM.conf.fit, movies.BPM.conf.pred))
```

```
##           2.5%    97.5%    mean    2.5%    97.5%    pred
## [1,] 48.00963 46.62389 49.39537 48.00963 28.24466 67.77460 48.00963
## [2,] 77.39261 76.01322 78.77200 77.39261 57.62808 97.15713 77.39261
## [3,] 82.43884 80.81128 84.06640 82.43884 62.65544 102.22223 82.43884
## [4,] 72.73853 71.14737 74.32968 72.73853 52.95809 92.51896 72.73853
## [5,] 40.64808 39.32988 41.96629 40.64808 20.88773 60.40843 40.64808
## [6,] 85.75164 83.87994 87.62335 85.75164 65.94667 105.55662 85.75164
```

```
# Extract a binary vector of zeros and ones for the variables included in the BPM
BPM = as.vector(which.matrix(movies.ZS.mcmc$which[movies.BPM$best], movies.ZS.mcmc$n.
vars))
BPM
```

```
## [1] 1 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0
```

```
# Re-run regression and specify `bestmodel` and `n.models`
movies.ZS.BPM = bas.lm(audience_score ~ ., data = moviesReg, prior = "ZS-null", model
prior = uniform(), bestmodel = BPM, n.models = 1)
```

```
# Obtain coefficients
coef(movies.ZS.BPM)
```

```
##
## Marginal Posterior Summaries of Coefficients:
##
## Using BMA
##
## Based on the top 1 models
##      post mean  post SD  post p(B != 0)
## Intercept      62.34769    0.39383    1.00000
## feature_filmyes    0.00000    0.00000    0.00000
## dramayes          0.00000    0.00000    0.00000
## runtime         -0.05353    0.02105    1.00000
## mpaa_rating_Ryes   0.00000    0.00000    0.00000
## thtr_rel_year      0.00000    0.00000    0.00000
## oscar_seasonyes    0.00000    0.00000    0.00000
## summer_seasonyes   0.00000    0.00000    0.00000
## imdb_rating       14.95802    0.57691    1.00000
## imdb_num_votes     0.00000    0.00000    0.00000
## critics_score      0.07025    0.02154    1.00000
## best_pic_nomyes     0.00000    0.00000    0.00000
## best_pic_winyes     0.00000    0.00000    0.00000
## best_actor_winyes   0.00000    0.00000    0.00000
## best_actress_winyes 0.00000    0.00000    0.00000
## best_dir_winyes     0.00000    0.00000    0.00000
## top200_boxyes      0.00000    0.00000    0.00000
```

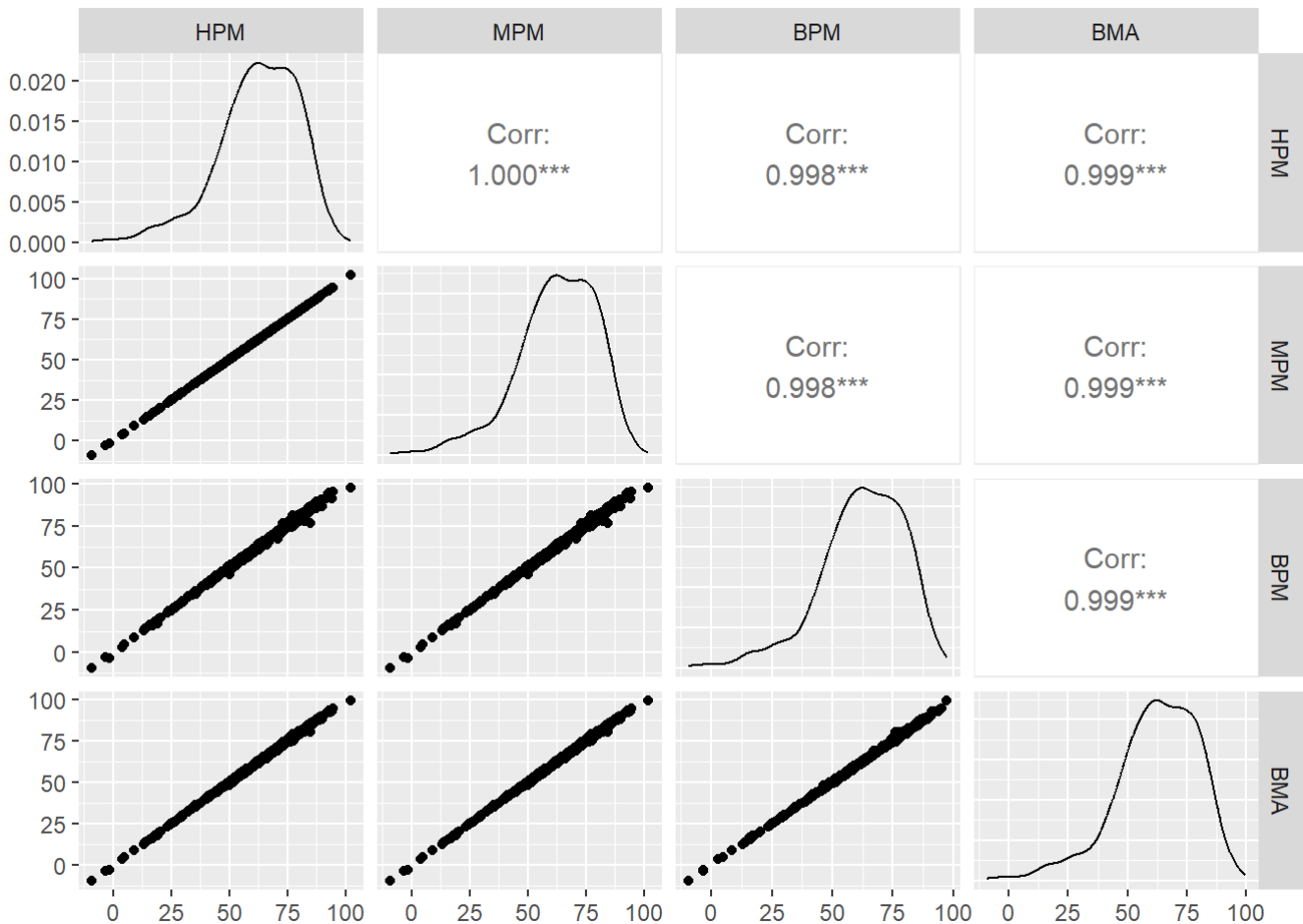
Comparison of Models

After discussing all 4 different models, let us compare their prediction results.

```
# Set plot settings
par(cex = 1.8, cex.axis = 1.8, cex.lab = 2, mfrow = c(2,2), mar = c(5, 5, 3, 3), col.
lab = "darkgrey", col.axis = "darkgrey", col = "darkgrey")
# Load library and plot paired-correlations
library(GGally)
```

```
## Registered S3 method overwritten by 'GGally':
##   method from
##   +.gg      ggplot2
```

```
ggpairs(data.frame(HPM = as.vector(movies.HPM$fit),
MPM = as.vector(movies.MPM$fit),
BPM = as.vector(movies.BPM$fit),
BMA = as.vector(movies.BMA$fit)))
```



From the above paired correlation plots, we see that the correlations among them are extremely high. The correlation value between any two models is almost one. This suggests that the models are in agreement and may be capturing similar underlying patterns in the data.

5. Prediction with new Data

Using the `newdata` option in the `predict` function, we can obtain prediction from a new data set.

An attempt is made to predict the audience score of the movie “Hidden Figures”. The movie had a limited release on December 25, 2016, and wide release on January 6, 2017. Putting the information in the data format:

```

Hidden_Figures <- data.frame(runtime = 127,
                             thtr_rel_year = 2016,
                             imdb_rating = 7.8,
                             imdb_num_votes = 263000,
                             critics_score = 74,
                             best_pic_nom = "yes",
                             best_pic_win = "no",
                             best_actor_win = "no",
                             best_actress_win = "no",
                             best_dir_win = "no",
                             top200_box = "yes",
                             feature_film = "yes",
                             drama = "yes",
                             mpaa_rating_R = "no",
                             oscar_season = "yes",
                             summer_season = "no")

BMA.new = predict(movies.ZS.mcmc, newdata = Hidden_Figures, estimator = "BMA", se.fit
= TRUE, nsim = 10000)
movies.conf.fit.new = confint(BMA.new, parm = "mean")
movies.conf.pred.new = confint(BMA.new, parm = "pred")
# Show the combined results compared to the fitted values in BPM
cbind(movies.conf.fit.new, movies.conf.pred.new)

```

```

##           2.5%    97.5%    mean    2.5%    97.5%    pred
## [1,] 80.09237 87.86898 83.12743 62.70155 103.3622 83.12743

```

We can see that the predicted value is about 83. The Rotten Tomatoes audience score for this is 93. So, the difference between actual and computed is about 10. However, we can see that the 95% confidence intervals of the predicted score covers this value.

6. Conclusion

Out of the sixteen predictors, it is found that only two or a maximum of three contribute to the predictions. This finding is consistent with all the models. Though the prediction is not accurate, the procedure followed is satisfactory. Perhaps a bigger data size may yield in better results.

References:

1. OpenIntro Statistics, Fourth Edition, - David Diez, Mine Cetinkaya-Rundel, Christopher D Barr.
2. An Introduction to Bayesian Thinking A Companion to the Statistics with R Course - Merlise Clyde, Mine Çetinkaya-Rundel, Colin Rundel, David Banks, Christine Chai, Lizzy Huan
3. Statistics and R - online course from edX